軟體工程 (Software Engineering)

第六課：使用者體驗 (UX) 設計與以人為本的 AI (UX Design & Human-Centered AI)

授課教師：薛念林 教授
資訊工程學系
逢甲大學

第六章：課程藍圖與核心主題

第一部分：UX 基礎與易用性啟發法則

- **6.1 使用者體驗 (UX) 基礎：** 什麼是 UX、諾曼門（Norman Doors）、三大不良設計原型、台灣健保快易通 App 改版案例、UI 與 UX 的本質差異、設計思考五大階段。
- **6.2 尼爾森 10 大易用性啟發法則：** 系統狀態可見度、符合真實世界、使用者控制與自由、一致性與標準、防錯設計、辨識取代回憶、彈性與效率捷徑、簡約美學、協助使用者修復錯誤、說明文件。

第二部分：AI 賦能設計與智慧介面

- **6.3 AI 賦能 UX 設計 (AI for UX)：** AI 作為設計放大器、RTCF 結構化提示詞工程框架、自動化啟發式稽核、多代理人協同原型打造。
- **6.4 為 AI 打造卓越體驗 (UX for AI)：** 為機率性非確定性 AI 設計介面、延遲與思考軌跡管理、人類自主掌控權、幻覺防護圍欄、可解釋性 AI (XAI)。
- **6.5 易用性評估與工程整合：** 尼爾森 0-4 級嚴重度評級、專家啟發式評估 vs. 真人使用者測試、章節回顧測驗與工程總結。

6.1 使用者體驗 (UX) 設計基礎

> *"Good design is actually a lot harder to notice than poor design, in part because good designs fit our needs so well that the design is invisible."*
> *(優秀的設計往往比糟糕的設計更難被察覺，因為好設計與我們的需求契合得天衣無縫，以至於它本身隱形了。)*
> *— Don Norman (《設計的心理學》作者)*

為什麼好設計如此重要？從「諾曼門」談起

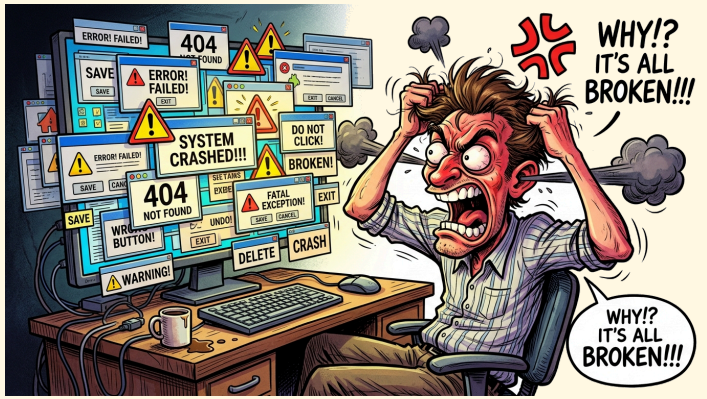
> *什麼是「諾曼門 (Norman Door)」？一扇外觀給出**錯誤直覺暗示**的門——例如裝了明顯的**「拉把」，實際上卻必須「用推的」**才打得開！*

- **當設計違背直覺時的真實情緒（尷尬與生氣）：**
  - 看到門把下意識用力拉，門卻紋風不動——瞬間讓人感到**尷尬**、**丟臉**、**懷疑自己，甚至惱怒生氣**。
  - Don Norman 指出：**這絕不是使用者的錯，而是糟糕的設計**違背了人類的本能反應！
- **從一扇門帶出 UX（使用者體驗）的核心價值：**
  - *「當一扇日常大門還需要貼上*『請用推的 (PUSH)』**字條時，這個設計本身就失敗了。」* —— Don Norman
  - 軟體介面同樣充滿「數位諾曼門」：好的 **UX 設計**讓視覺線索契合人類直覺，讓使用者感到**順暢與被尊重**，而非受挫生氣。

劣質 UI/UX 所帶來的挫折與代價

> *「當介面混亂難用時，使用者很少怪罪系統架構師——他們只會感到**認知挫折**、懷疑自己，然後**憤而放棄產品**。」*

- **糟糕軟體對使用者心理的摧殘：**
  - 引發認知超載、困惑迷惘、焦慮無助，最後導致使用者憤而放棄（User Rage & Churn）。
  - 使用者往往不會怪罪背後的資料庫或架構師，只會留下 1 星負評並怒刪 App。
- **劣質 UX 如何摧毀企業商業價值：**
  - **購物車放棄率高達 70%：** 結帳流程過於繁瑣、表單驗證不明確直接導致營收蒸發。
  - **龐大客服成本支出：** 數以萬計的客服電話與工單，根本源頭都是混亂的介面導覽。
  - **企業員工生產力內耗：** 難用的內部企業 ERP/CRM 系統，導致日常操作頻繁出錯與嚴重工時浪費。



不良設計原型 (1)：「夜市擺攤風」的感官超載

> *「夜市擺攤風」(Arngren-Style Overload)：將數百項商品、閃爍跑馬燈與衝突連結全部硬塞進單一畫面，毫無視覺階層。*

- **為什麼這種設計很糟糕？**
  - **模式識別失效：** 每個商品區塊的長寬比、對齊線與字體完全隨機，大腦無法建立模式，必須對每個點重新「對焦」，造成極度疲勞。
  - **缺乏網格系統與視覺錨點：** 沒有水平或垂直對齊軸線，視線無法順暢掃描。
  - **急性感官超載：** 當畫面上每個元素都同時大聲搶奪注意力時，使用者無法找到核心任務，跳出率極高。

https://www.mockplus.com/blog/post/bad-web-design

不良設計原型 (2)：「油漆不用錢」的色彩對比災難

> *「油漆不用錢」原型： 濫用衝突刺眼的配色、波紋背景與浮雕陰影，徹底摧毀易讀性與專業品牌信賴感。*

- **為什麼這種設計很糟糕？**
  - **色彩對比崩壞：** 在藍色波紋漸層背景上直接壓深紅細體字與亮黃字，產生視覺閃爍感，嚴重違反 **WCAG 無障礙對比度標準**。
  - **排版與字體災難：** 標題濫用厚重浮雕陰影增加視覺噪音；連結列表長短參差不齊，造成嚴重視覺疲勞。
  - **缺乏負空間且摧毀信賴感：** 內容塞得密不透風，尤其在專業醫療 X 光器材網站採用這種配色，會直接摧毀專業信任度。

https://www.mockplus.com/blog/post/bad-web-design

不良設計原型 (3)：「不知從何下手」的導覽與行寬失控

> *「不知從何下手」原型：滿螢幕充斥著數十個尺寸相同、未分主次的按鈕與超寬文字段落，缺乏明確的行動呼籲 (CTA)。*

- **為什麼這種設計很糟糕？**
  - **缺乏視覺焦點與主次：** 左側一整排紅黑按鈕尺寸與視覺權重完全相同，沒有主要行動呼籲（CTA），使用者不知道該先按哪裡。
  - **資訊洪流無分類：** 頂部將電話、地址與標語全塞進刺眼的亮綠橫幅，毫無導覽層次。
  - **文字行寬失控 (Line Length)：** 右側段落直接橫跨整個寬螢幕，遠超過人類舒適閱讀的 **每行 45～75 字元** 上限，強迫眼球大幅擺動。

https://www.mockplus.com/blog/post/bad-web-design

真實案例互動：健保快易通 App — 猜猜口罩去哪裡預約？

> *課堂挑戰：如果你想預約購買口罩，猜猜看應該點擊哪一個圖示（1–6）？*

第一版緊急上線：猜猜在哪裡買口罩 (1–6)？

1. 口罩查詢 ｜ **2.** 健康存摺 ｜ **3.** 改革日記
4. 院所查詢 ｜ **5.** 行動櫃檯 ｜ **6.** 更多…

解答：竟藏在 2 號「健康存摺」！→ 改版後九宮格

為何放「健康存摺」？因為裡面才有登入驗證模組！
改版後：直接將 **eMask 口罩預購** 獨立拉到第一排正中央。

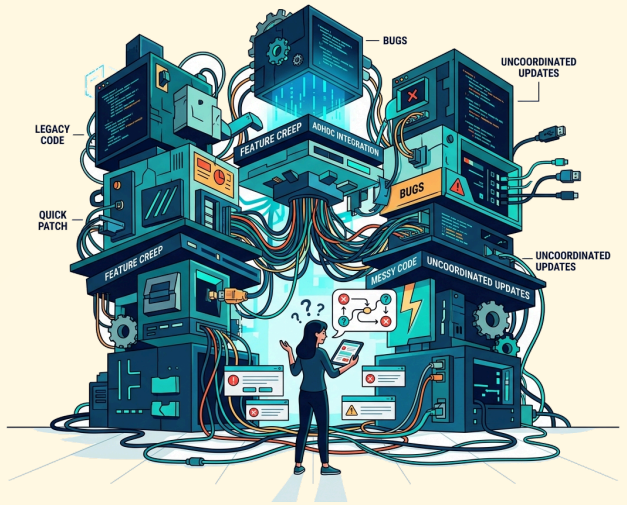
案例剖析：工程師後端思維 vs. 使用者心智模型

- **第一版緊急上線為何讓民眾找不到？（以系統實作驅動 UI 的反模式）：**
  - **遷就後端認證模組（康威定律）：** 因為買口罩需要實名登入，工程師直接把功能掛在已經有登入機制的 **2 號「健康存摺」** 內，完全違背民眾對「存摺＝查就醫紀錄」的認知。
  - **誤導性的示能線索 (False Signifier)：** **1 號「口罩查詢」** 畫著戴口罩的圖示，吸引無數民眾誤點進入死胡同；頂部紅字跑馬燈看起來像純文字廣告，看不出可以點擊。
  - **視覺規格不一致：** 圓形、長方形存摺、立體書本混雜，缺乏統一網格與階層。
- **改版後如何解決問題？（以使用者目標驅動的九宮格重構）：**
  - **核心任務直接浮出表面：** 將 **「eMask 口罩預購」**、**「History 購買紀錄」** 與 **「Mask Map 口罩地圖」** 三大防疫任務直接獨立拉到最上排第一層。
  - **標準化視覺與雙語標籤：** 統一採用圓形圖示與整齊九宮格，並加上中英雙語標示，讓使用者一秒找到目標。

為什麼做好 UX 那麼難？「知識的詛咒」

> ***UX 至高鐵律：***「*你不是你的使用者——千萬別因為自己親手寫了後端資料庫，就誤以為初次面對畫面的使用者也看得懂。*」

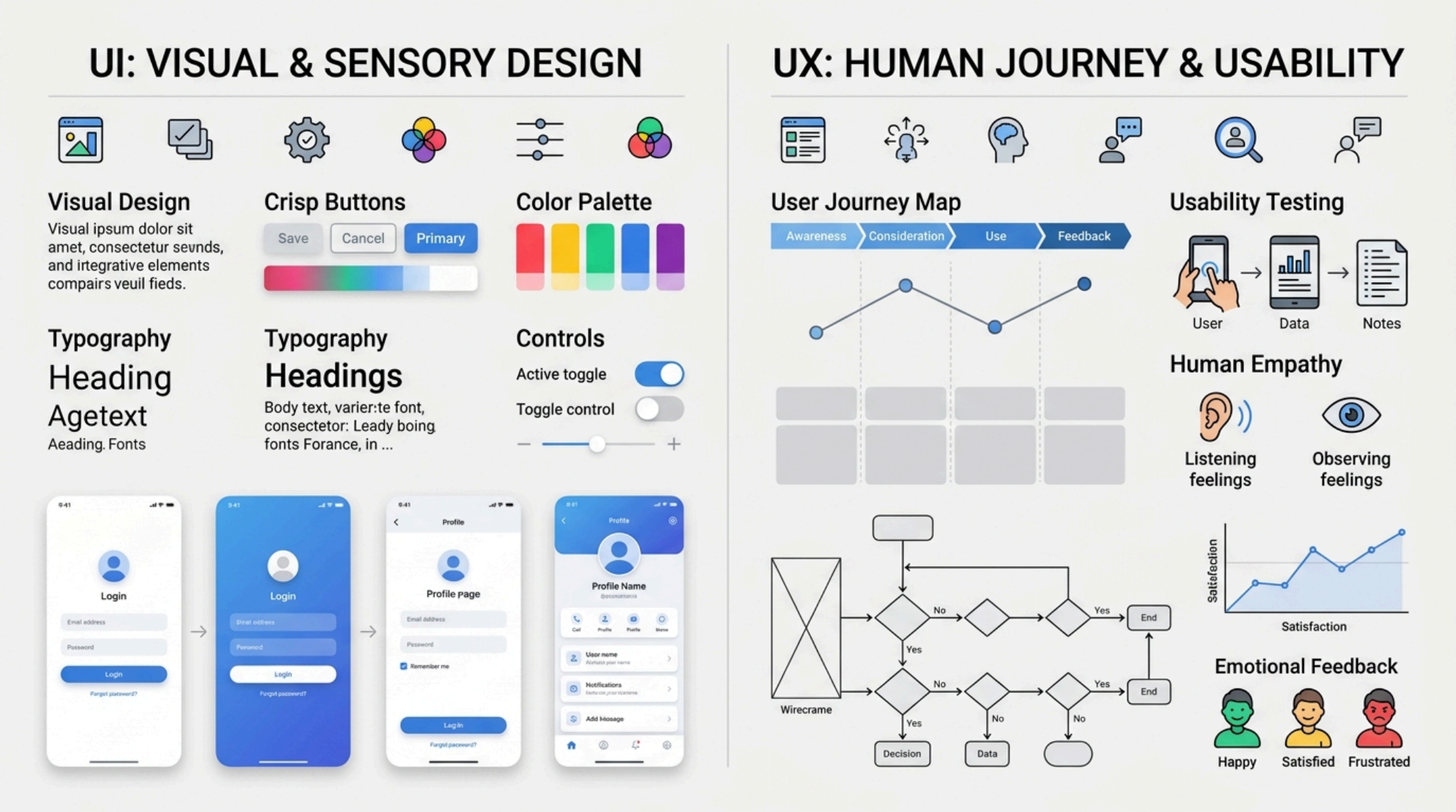
- **1.「你不是你的使用者」 (You Are Not The User)：**
  - 軟體工程師擁有深厚的技術心智模型（關聯式綱要、REST API 路由、多執行緒）。
  - 一般使用者只具備日常生活的心智模型（訂晚餐、掛號、查帳單）。
- **2. 知識的詛咒 (The Curse of Knowledge)：**
  - 工程師自己辛苦花三個月把功能刻出來，對介面瞭若指掌，因此覺得一切操作都「極度理所當然、非常直覺」。
  - 工程師的大腦已經無法體會初次面對該介面時的陌生與困惑。
- **3. 防衛心理 vs. 同理心思考：**
  - 工程師往往依照後端資料表開畫面，要求使用者按照資料庫的邏輯填表單。



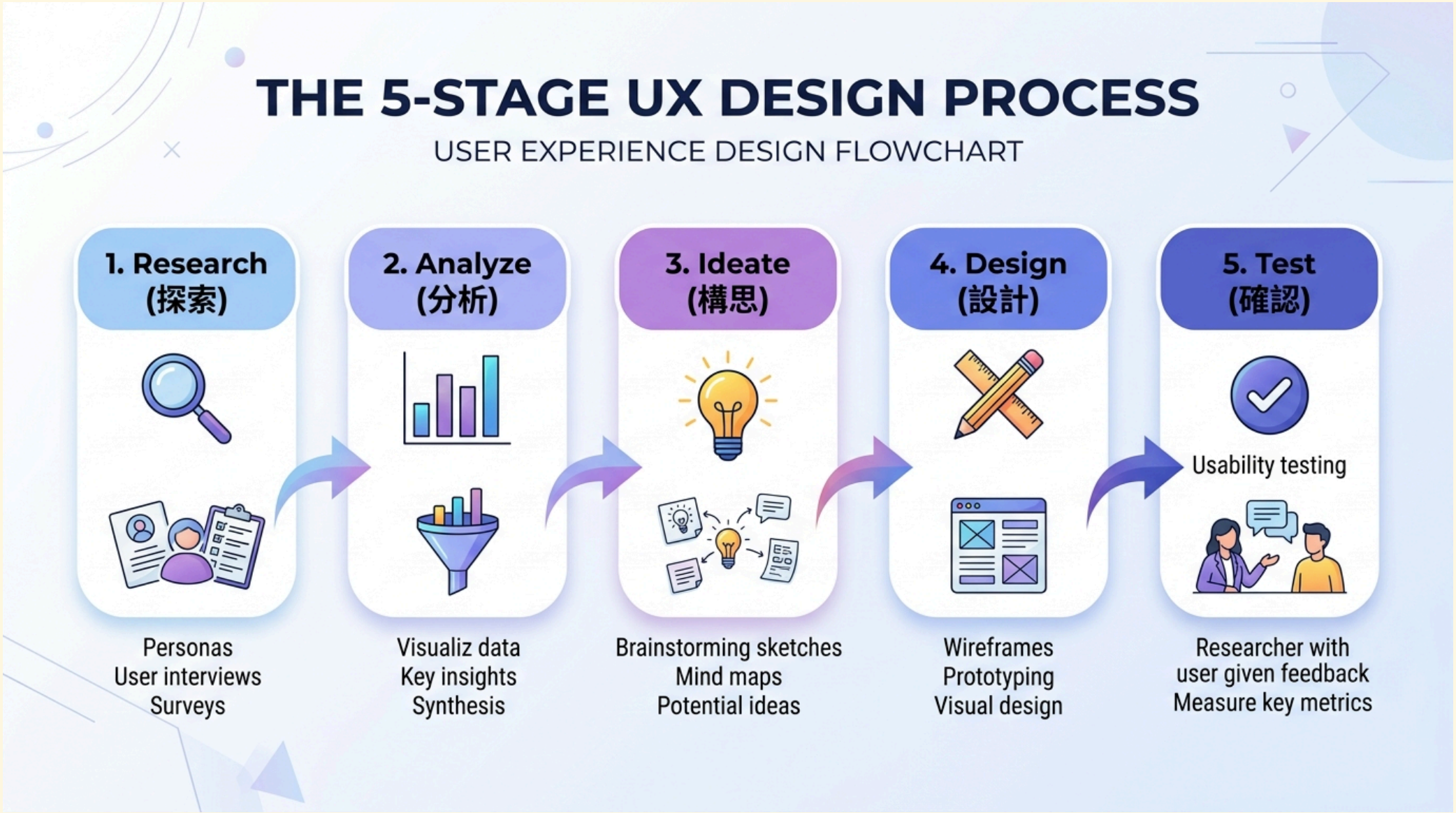
UI (使用者介面) vs. UX (使用者體驗)

- **使用者介面 (UI / User Interface)：**
  - 人類與軟體互動時，所看見、聽見與觸摸的具體感官接觸點。
  - *元素涵蓋：* 按鈕外觀、字型排版、色彩配置、邊距間距、CSS 格線、圖示、過渡微動畫。
  - *核心關切：* 「這個畫面在視覺上看起來如何？渲染與動效順不順暢？」
- **使用者體驗 (UX / User Experience)：**
  - 人類與品牌、服務或系統在完整旅程中產生的整體心理與情感認知感受。
  - *元素涵蓋：* 任務達成效率、流程阻力、認知負擔、滿意度、容錯修復能力與信賴感。
  - *核心關切：* 「使用者是否能直覺、高效且愉悅地達成他們的目標？」
- **經典生動比喻：**
  - UI 是精緻的馬鞍、韁繩與馬鐙；UX 則是騎在馬背上奔馳的整體駕馭感受！

視覺化比對：UI 與 UX 的互動架構解剖



設計思考 5 大階段 (The 5-Stage Design Thinking)



設計思考五大階段詳細拆解

- **1. Empathize (同理 / 探索)：**
  - 進行質化訪談、現地情境觀察（Contextual Inquiry）與使用者行為追蹤。
  - 挖掘未被說出口的深層挫折與真實心理痛點。
- **2. Define (定義問題)：**
  - 將質化研究收斂為具象的人物誌（Personas）、同理心地圖與精確的 "How Might We (我們該如何...)" 問題陳述。
- **3. Ideate (構思解法)：**
  - 開展多元創意思考，暫緩評判；快速繪製低保真紙上草圖（Paper Wireframes）。
- **4. Prototype (打造原型)：**
  - 製作可供點擊測試的實體構件（Figma 可互動點擊原型、可運行的 HTML/CSS 假資料畫面）。
- **5. Test (驗證測試)：**
  - 將原型交由真實目標使用者操作，觀察卡關點，量化任務達成率並快速迭代修正。

概念核對問答 1 (CCQ 1)：UI vs. UX 與同理心落差

一款行動網銀 App 具備精緻的毛玻璃動效（**出色的 UI**），但卻有 **68% 的新戶在轉帳時放棄**，因為驗證流程直接顯示資料庫代碼（`TXN_ROUTING_ID`）。此失敗的真正根源與 **設計思考 (Design Thinking)** 解法為何？

- **A. UI 視覺渲染缺陷**；應將毛玻璃特效替換為高對比度的 Material Design 按鈕元件。
- **B. 後端吞吐量瓶頸**；應導入 Redis 快取層，確保轉帳查詢在 50 毫秒內完成。
- **C. UX 與同理心落差失敗**；須調研使用者痛點並依顧客心智模型重構轉帳原型。
- **D. 使用者訓練不足**；應強制規定新戶在首次轉帳前先看完 15 分鐘教學影片。

6.2 尼爾森 10 大易用性啟發法則

> *"Jakob's Law: Users spend most of their time on other sites. This means that users prefer your site to work the same way as all the other sites they already know."*
> *(雅各布定律：使用者絕大部分的時間都在別人的網站上。這意味著他們強烈偏好你的網站運作方式，與他們早已熟知的所有其他網站完全一樣。)*
> *— Jakob Nielsen (Nielsen Norman Group 創辦人)*

概覽：尼爾森 10 大易用性啟發法則 (Overview)

- **NS01. 系統狀態可見度：** 透過及時適當的回饋，讓使用者隨時掌握系統狀態。
- **NS02. 符合真實世界：** 採用使用者熟悉的語言詞彙、概念與實體隱喻。
- **NS03. 使用者控制與自由度：** 提供清晰明確的「緊急出口」（復原 Undo / 重做）。
- **NS04. 一致性與標準性：** 遵循平台與產業通用規範（雅各布定律）。
- **NS05. 防錯設計：** 事前透過審慎設計消除容易出錯的情境，勝過事後道歉。
- **NS06. 辨識取代記憶：** 將物件、動作與選項視覺化，減輕大腦記憶負擔。
- **NS07. 彈性與使用效率：** 提供捷徑（Accelerators）滿足高頻專業進階用戶。
- **NS08. 簡約美學設計：** 消除無關雜訊，將訊噪比（Signal-to-Noise）最大化。
- **NS09. 協助辨識與修復錯誤：** 用白話清楚說明問題，並建設性指出解法。
- **NS10. 說明文件與求助支援：** 提供容易檢索、任務導向且情境化的即時引導。

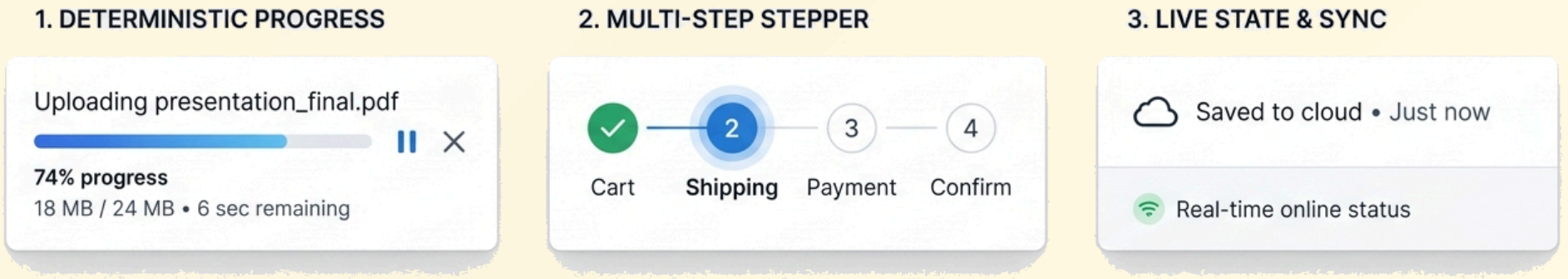
NS01: 系統狀態可見度 (Visibility of System Status)

- **啟發原則定義：**

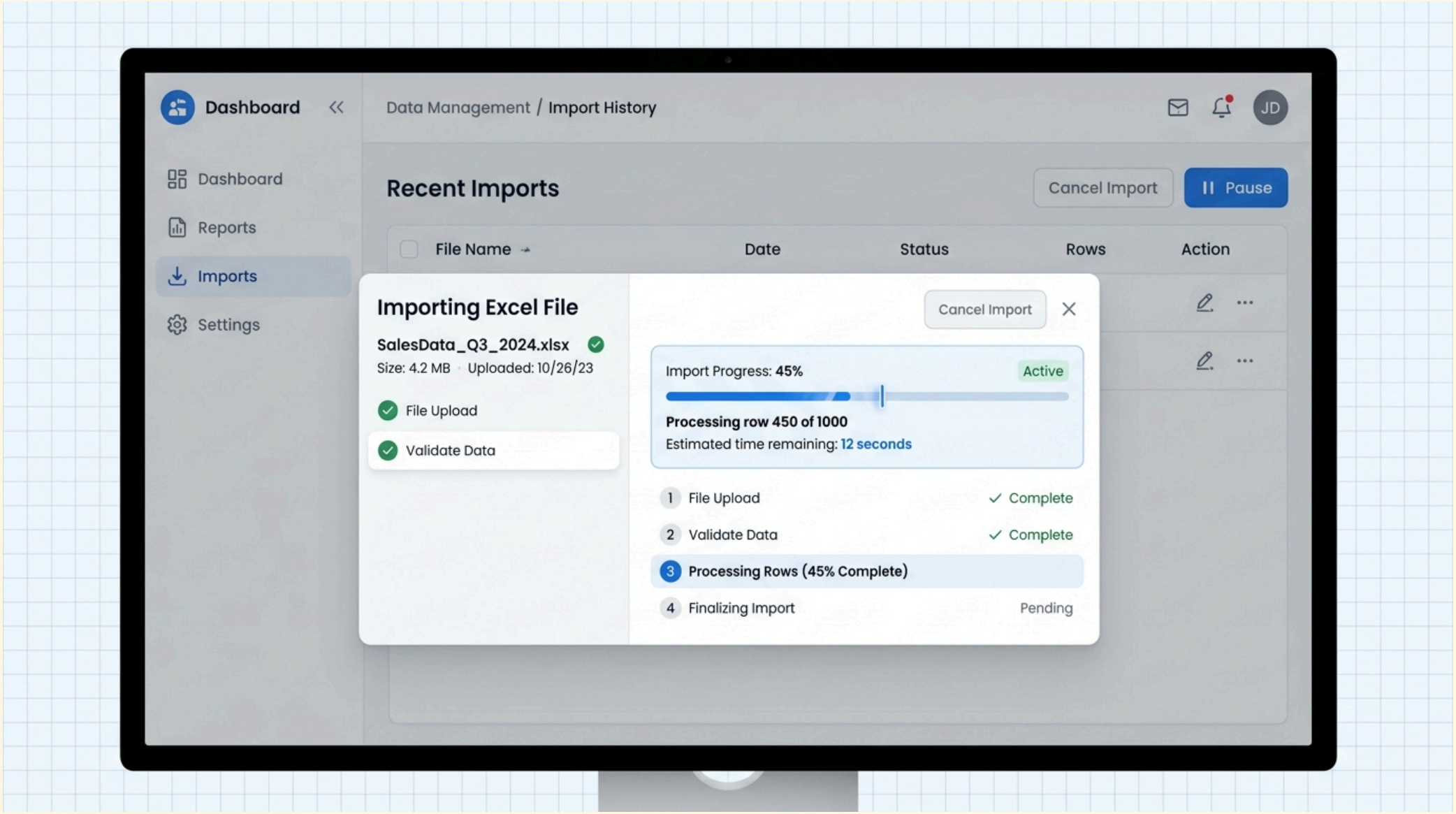
> *「系統應在**合理的時間內**，透過**適當的回饋 (Feedback)**，始終讓**使用者明確掌握**當前正在發生什麼事。」*

- **核心互動設計模式：**
  - **即時觸控回饋：** 按鈕點擊時的下沉態、微動效波紋（Ripple Effect），立即向使用者確認「輸入已被系統接收」。
  - **進度透明化：** 顯示具備確切百分比的進度條（%）、結帳步驟指示器（1. 購物車 → 2. 配送資訊 → 3. 付款）。
  - **背景任務即時狀態：** 旋轉進度環搭配具備動態文字的說明（*「正在上傳第 3 張相片，共 5 張...」*）。
  - **連線狀態提示：** 離線斷網即時警告橫幅、雲端同步狀態勾選指示。

NS01 實踐 (1/2)：進度條、步驟指示器與即時同步狀態



NS01 實踐 (2/2)：批次檔案匯入進度與預估剩餘時間



NS02: 符合真實世界 (Match System & Real World)

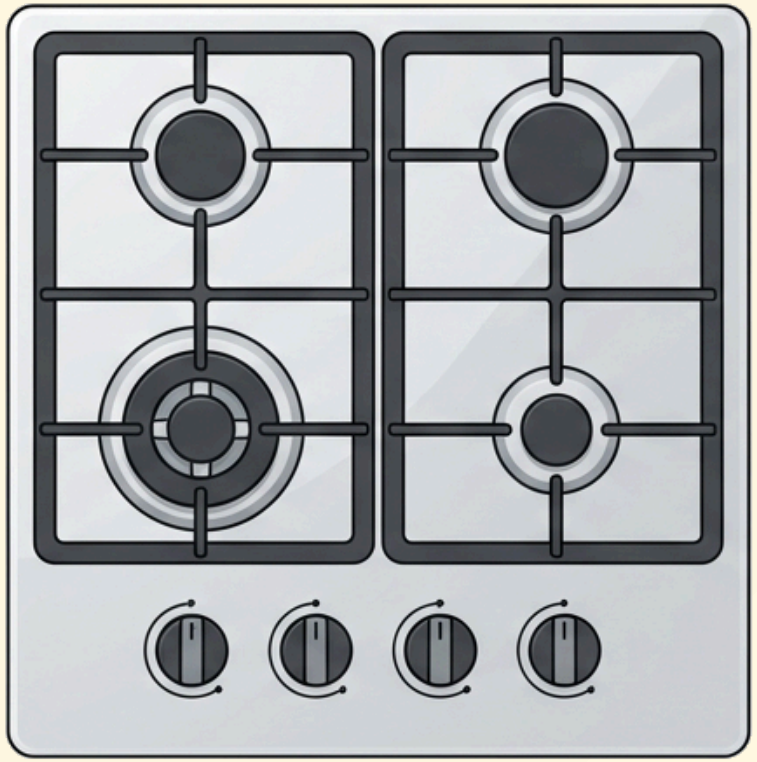
- **啟發原則定義：**

> *「系統應使用使用者的語言，採用使用者熟悉的詞彙、語句和概念，而不是系統內部的術語。遵循真實世界的通用慣例。」*

- **自然空間映射 (Natural Mapping)：**
  - 操作控制器在空間上的配置，應精確對應於被控制物體的物理空間幾何。
  - *經典反面教材：* 四口瓦斯爐排列為 2x2 方陣，但四個開關旋鈕卻排成一整條直線，強迫大腦進行繁複的空間座標換算，經常開錯爐火。
  - *優良設計：* 將四個旋鈕同樣排列為 2x2 矩陣，操作直覺秒懂且零出錯。
- **生活實體隱喻 (Familiar Metaphors)：**
  - 資源回收桶代表刪除、資料夾代表目錄階層、購物車代表待結帳商品。

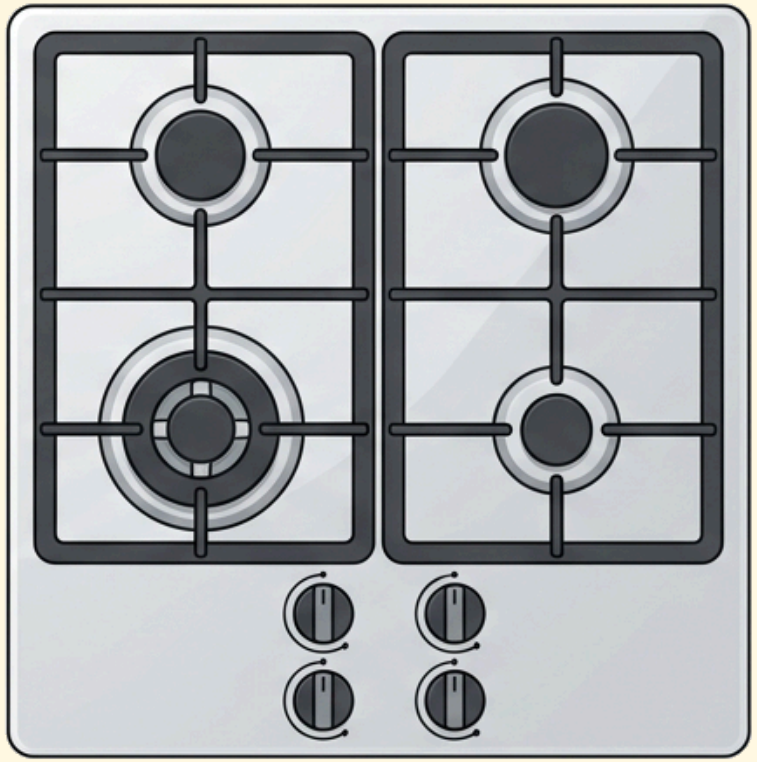
NS02 實踐 (1/2)：控制項的自然空間幾何映射

Unnatural Mapping (Arbitrary)



High Cognitive Friction — Which knob controls which burner? Requires labels and mental translation.

Natural Mapping (Spatial Correspondence)

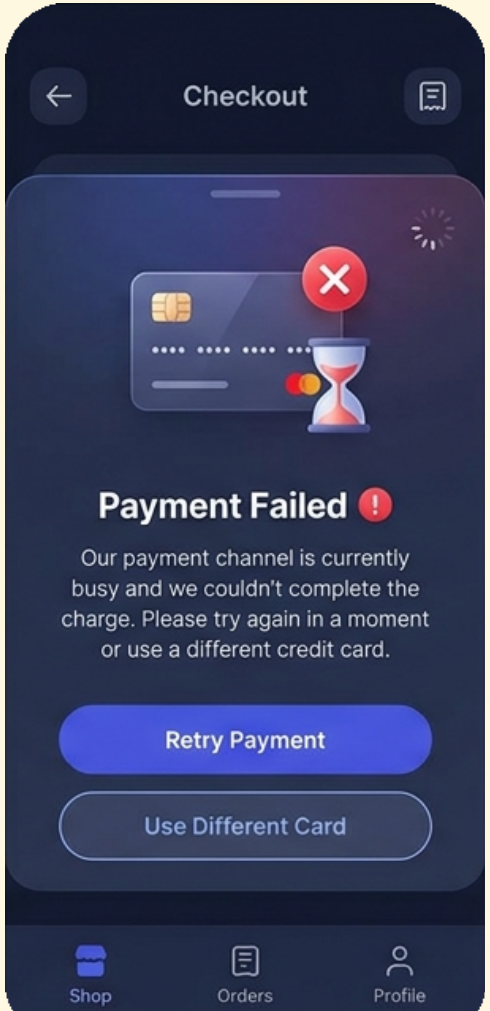


Intuitive Operation — Subconscious, error-free physical mapping.

NS02 實踐 (2/2)：使用日常語言取代生硬的系統代碼

> *為什麼重要？將後端例外代碼（`HTTP 502`、`ERR_SOCKET_TIMEOUT`）直接丟給消費者，不僅讓人一頭霧水，更會引發「到底有沒有被重複扣款？」的恐慌。*

- **系統導向的工程術語（不良設計）：**
  - ❌ *「Transaction Exception 403: Acquirer socket connection timed out.」*
  - 使用者陷入焦慮：*「我的信用卡到底刷過了沒？」*
- **以人為本的日常語言（優良設計）：**
  - ✅ **安心狀態確認：** 明確告知**「尚未完成扣款 (couldn't complete the charge)」**。
  - ✅ **生活化詞彙：** 以「目前付款通道繁忙」取代網路連線逾時術語。
  - ✅ **清楚下一步：** 提供直覺按鈕（`重試付款` 或 `改用其他信用卡`）。



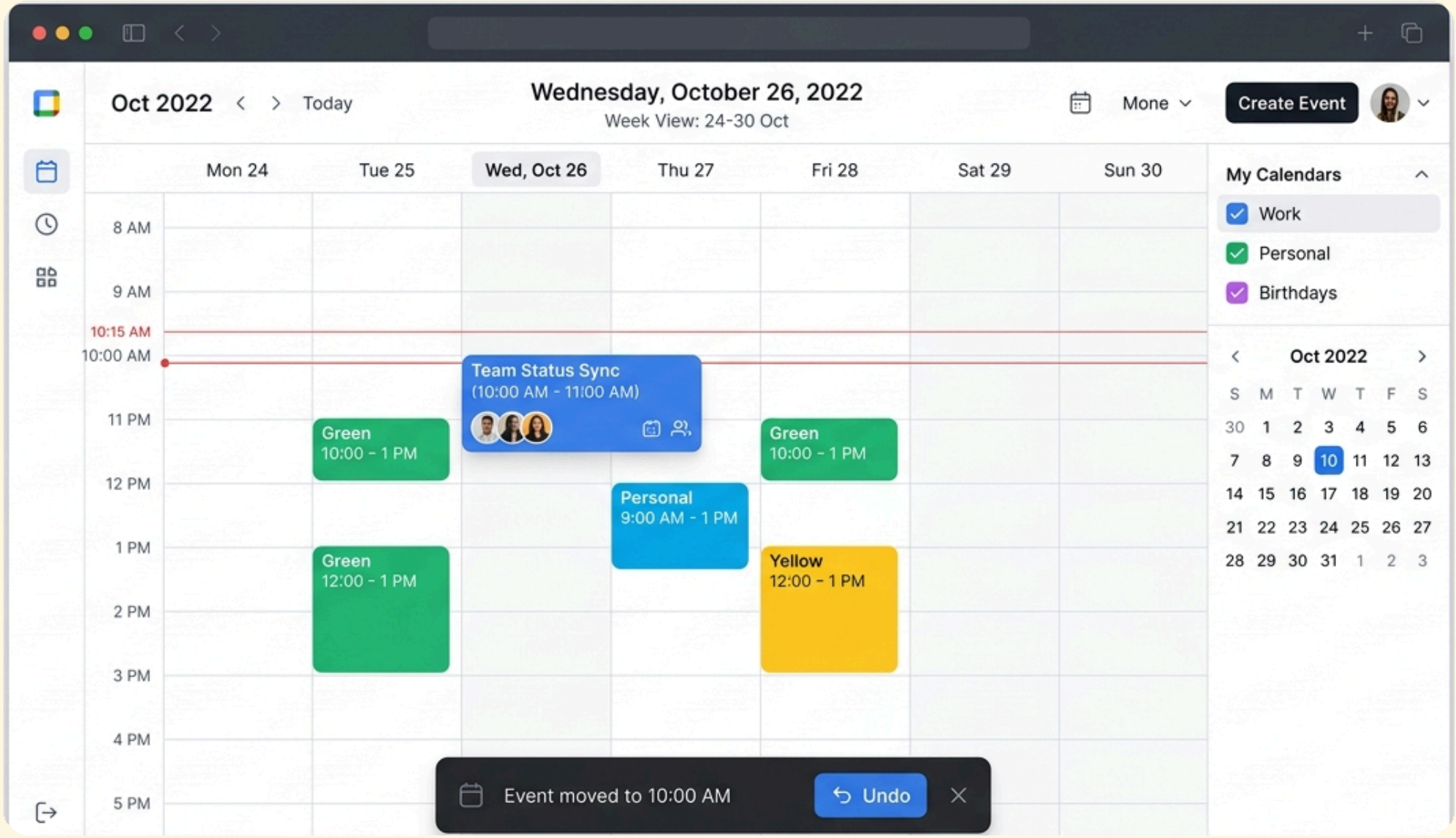
NS03: 使用者控制與自由度 (User Control & Freedom)

- **啟發原則定義：**

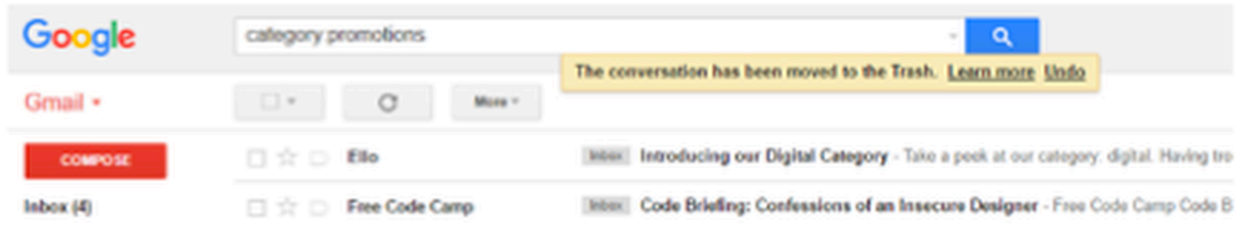
> *「使用者常因**誤操作**而誤入非預期的功能狀態，此時需要一個清晰標示的**『緊急出口 (Emergency Exit)』，**讓他們能**迅速脫離該狀態**，而不需經歷冗長的對話流程。」*

- **必備的工程實踐模式：**
  - **全域復原 / 重做 (Universal Undo / Redo)：** 編輯器、畫布工具及破壞性狀態變更均應全面支援 `Ctrl+Z` / `Cmd+Z`。
  - **清晰的取消出口：** 彈跳視窗（Modals）、檔案上傳與批次處理中，必須提供高對比、易於點擊的「取消 (Cancel)」按鈕。
  - **非破壞性導覽：** 允許使用者隨時退出多步驟結帳或註冊精靈，且不應擅自抹煞已填寫的表單暫存資料。
  - **資源回收暫存區 (Soft Deletes)：** 刪除操作先移至垃圾桶，保留 30 天復原期後才永久清除。

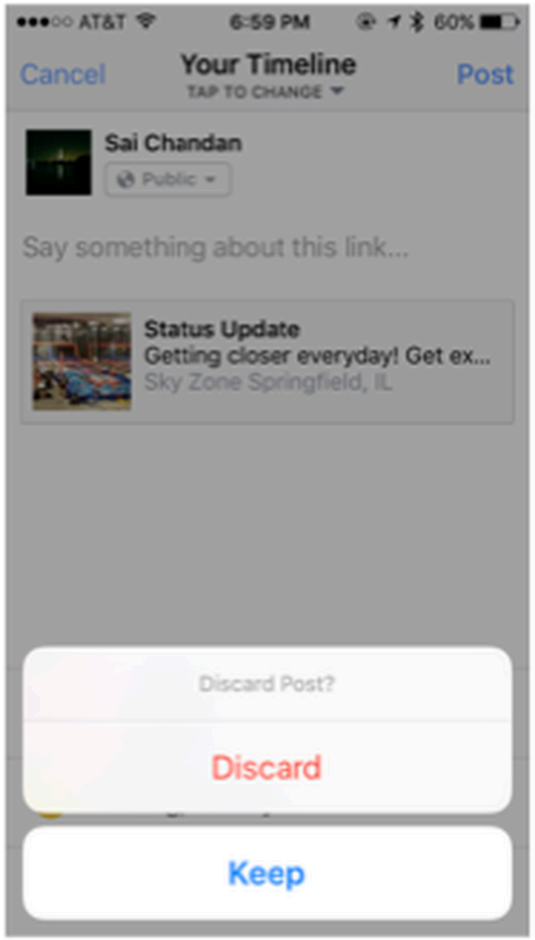
NS03 實踐 (1/2)：行事曆拖曳變更後的即時「復原 (Undo)」



NS03 實踐 (2/2)：信件刪除反悔機制與放棄編輯出口



👍 當你把信件丟到垃圾桶，系統提供你 **反悔** 的機會。甚至於是寄信的當下，也可以 Undo。



👍 放棄留言時，系統提供你 **取消** 的機會

https://medium.com/.../10-usability-heuristics-with-examples-4a81ada920c

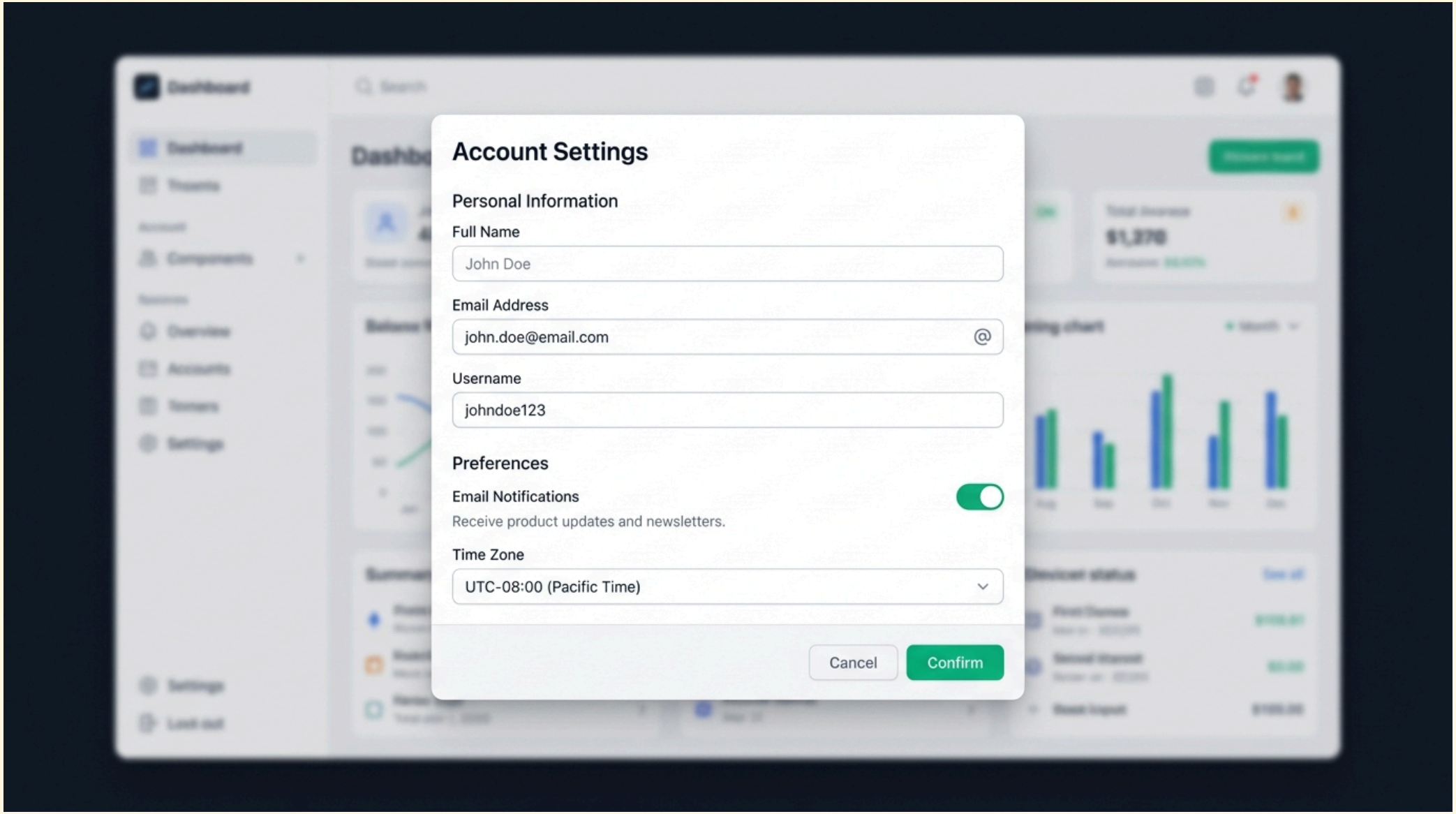
NS04: 一致性與標準性 (Consistency & Standards)

- **啟發原則定義：**

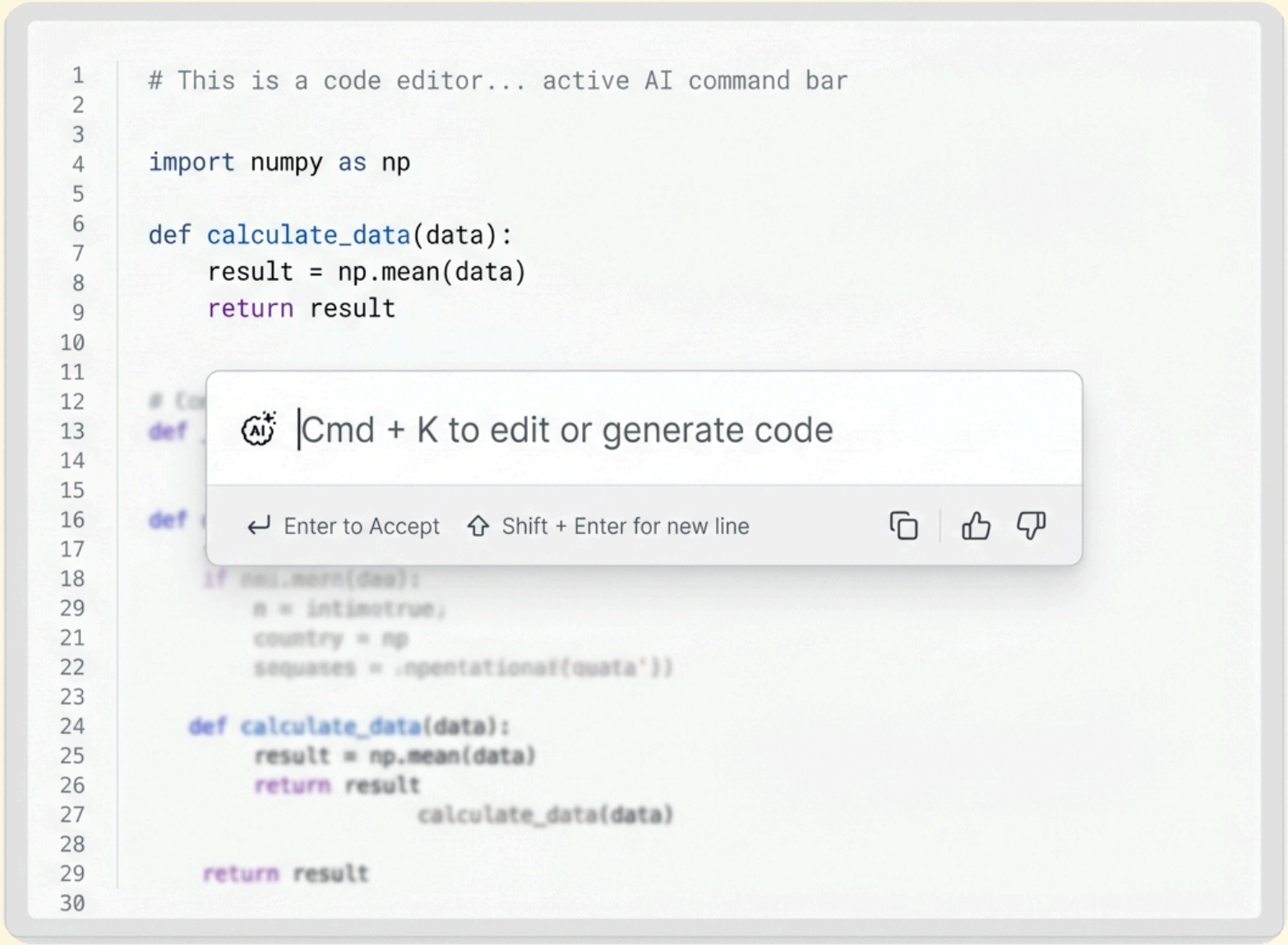
> *「使用者不應懷疑****不同的詞彙、狀態或操作是否代表相同的事物。應遵循平台與產業通用的既定慣例。」*

- **網際網路體驗的「雅各布定律 (Jakob's Law)」：**
  - 你的使用者有 99% 的數位生命都在**別人的網站與 App 上度過**。
  - 他們預期你的軟體運作方式，與他們熟知的其他產品完全一致（例如：左上角 Logo 回首頁、右上角購物車、放大鏡代表搜尋）。
- **一致性的兩大工程維度：**
  - **內部一致性 (Internal Consistency)：** 產品內部統一的設計系統（Design Tokens）、按鈕樣式、字級比例與名詞定義。
  - **外部一致性 (External Consistency)：** 遵循作業系統平台人機介面指南（iOS HIG 與 Android Material Design）。

NS04 實踐 (1/2)：一致的表單控制項與按鈕視覺階層



NS04 實踐 (2/2)：遵循跨平台標準慣例的快捷指令列




```
# This is a code editor... active AI command bar

import numpy as np

def calculate_data(data):
    result = np.mean(data)
    return result
```

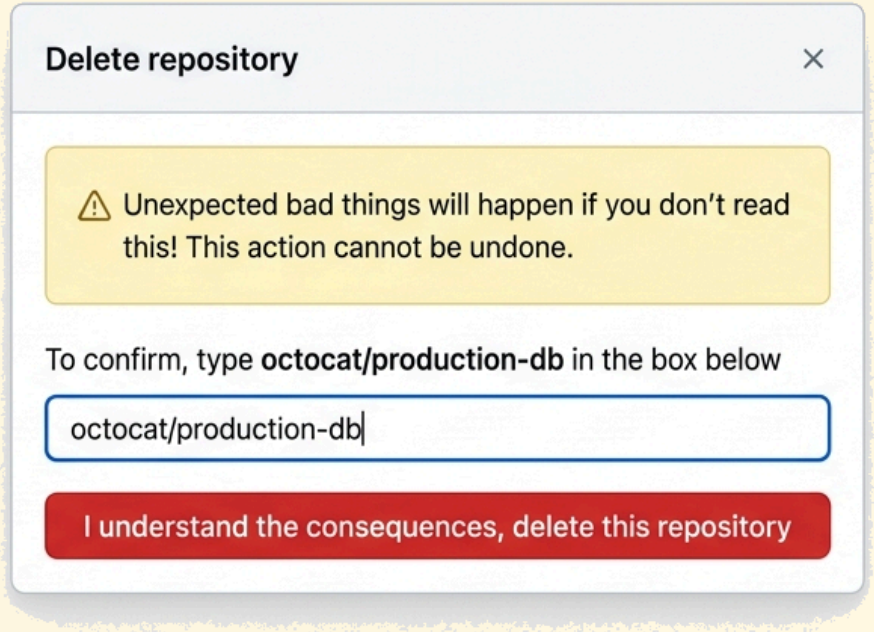
NS05: 防錯設計 (Error Prevention)

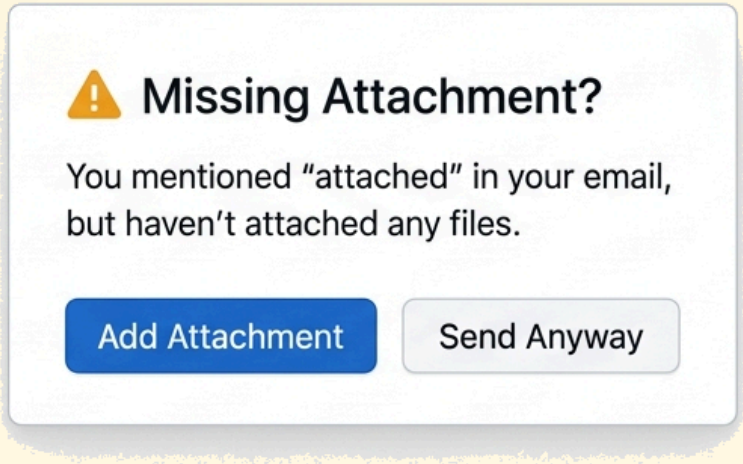
- **啟發原則定義：**

> *「比起再怎麼友善詳盡的錯誤訊息，更卓越的工程設計是透過**審慎的防呆設計**，在第一時間就防止錯誤發生。」*

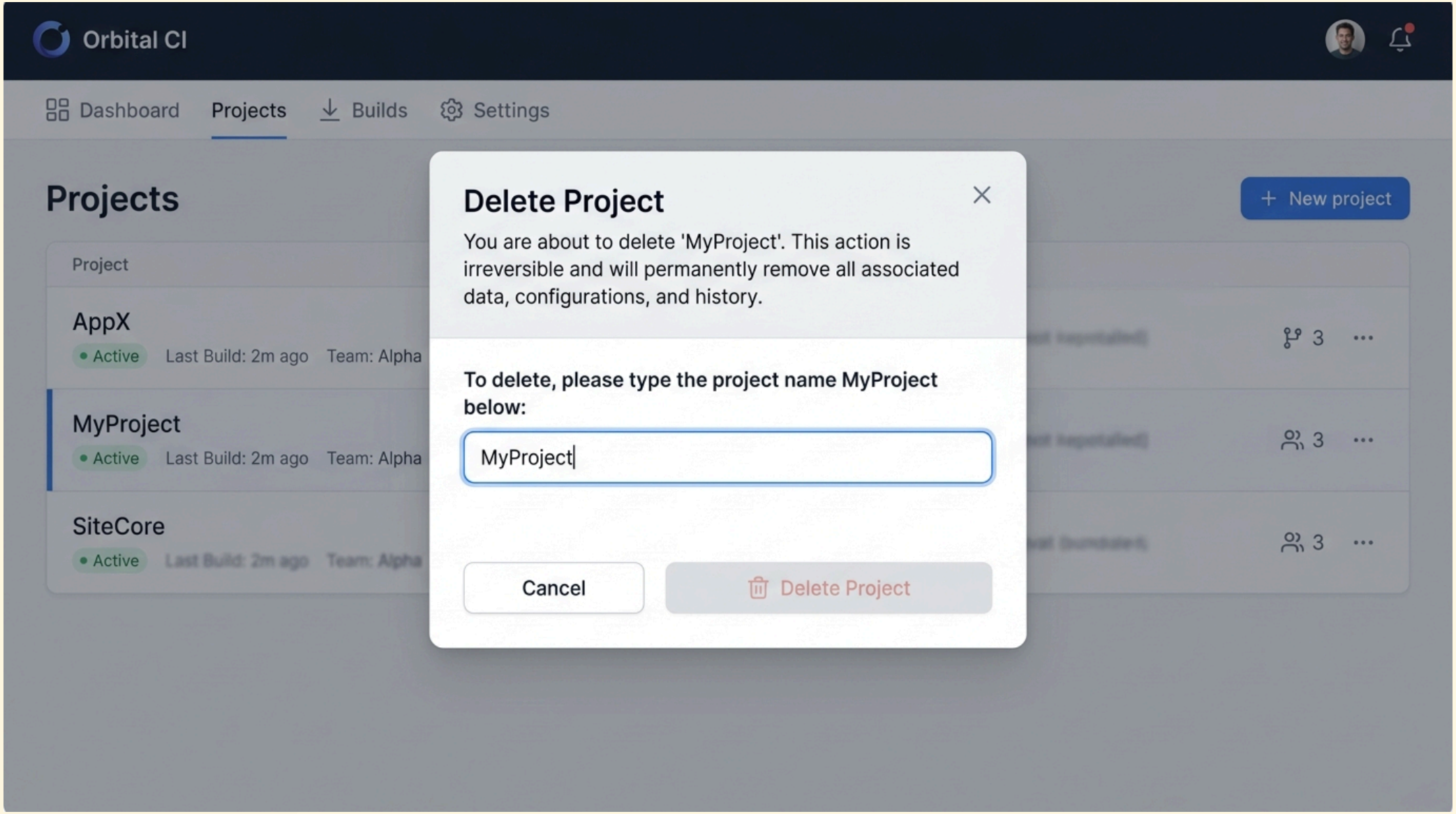
- **主動防禦型設計模式：**
  - **消除容易出錯的情境：** 在必填欄位尚未通過即時驗證之前，將「送出」按鈕保持反灰禁用狀態。
  - **輸入約束與限制 (Input Constraints)：** 使用可視化日期選擇器（Date Picker）取代純文字輸入框，徹底杜絕日期格式輸入錯誤（如 `YYYY/MM/DD` vs `DD/MM/YYYY`）。
  - **針對不可逆破壞性操作引入高阻力確認：** 刪除 GitHub 儲存庫前，要求使用者手動鍵入專案全名以喚醒意識。
  - **智慧預設值 (Intelligent Defaults)：** 預填最常見的合理參數，減少空表單帶給使用者的心理壓力。

NS05 實踐 (1/2)：破壞性操作防呆與「忘記附件」主動攔截





NS05 實踐 (2/2)：未輸入完整專案名稱前禁用刪除按鈕



NS06: 辨識取代記憶 (Recognition Rather Than Recall)

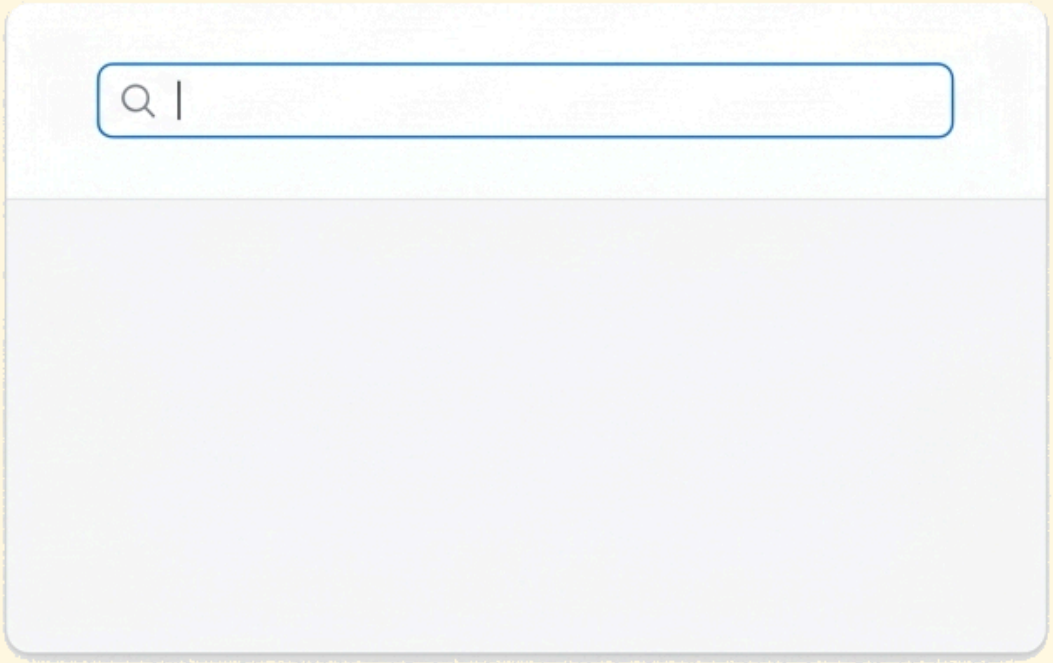
- **啟發原則定義：**

> *「透過將物件、操作和選項清晰可視化，使使用者的記憶負擔降到最低。使用者不應被迫記憶從對話流程某一部分到另一部分的資訊。」*

- **認知心理學基石：**
  - **辨識 (Recognition / 大腦輕鬆)：** 看到熟悉的視覺線索時能立刻認出（如同選擇題）。
  - **回憶 (Recall / 大腦耗能)：** 在毫無線索提示下，全憑記憶提取資訊（如同純文字終端機指令或申論題）。
- **UI 工程實踐：**
  - 搜尋輸入框點擊時自動展開最近搜尋歷史關鍵字與熱門建議。
  - 檔案列表顯示縮圖預覽，而非只有冰冷抽象的二進位檔名。
  - 文字選取時自動在游標旁邊浮現情境式編輯工具列。

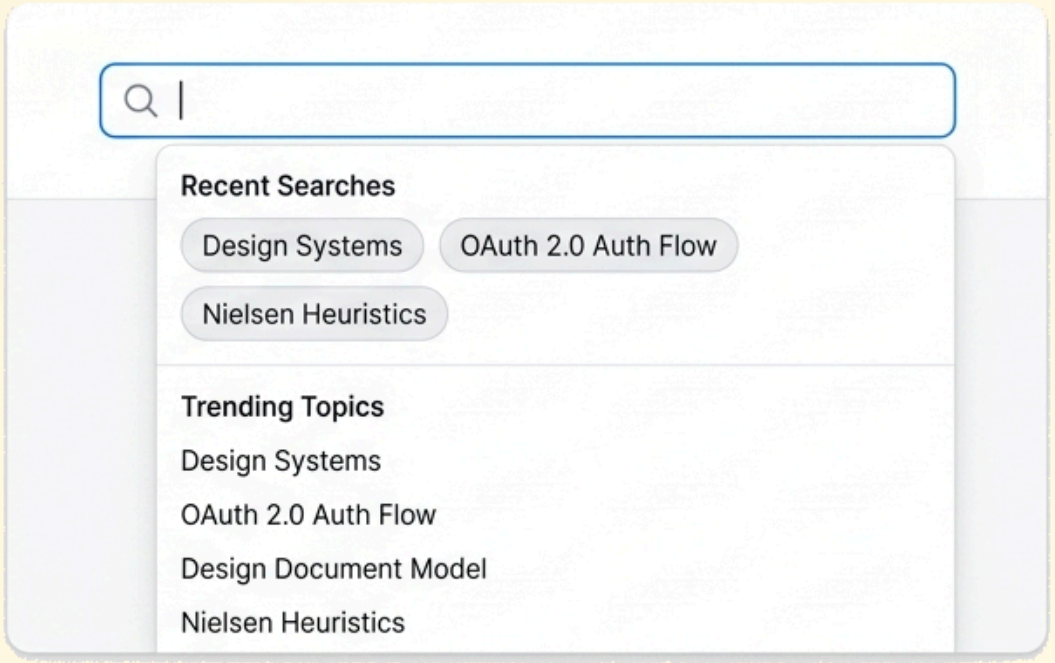
NS06 實踐 (1/2)：強迫記憶 (Recall) vs. 搜尋歷程辨識

Pure Recall (High Cognitive Load)



Forced Recall — Requires mental effort to retrieve past searches from memory.

Effortless Recognition (Low Cognitive Load)

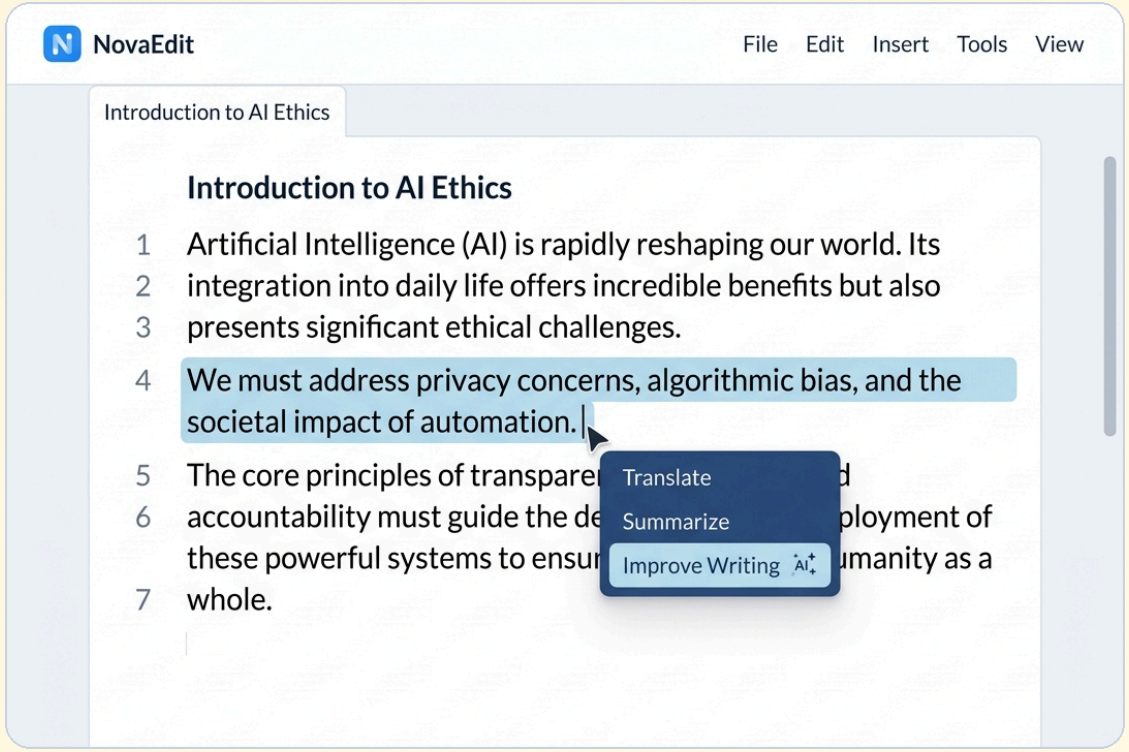


Recognition Over Recall — Past queries and suggestions are immediately visible for 1-tap selection.

NS06 實踐 (2/2)：情境式工具列如何消除「記憶負擔」

> *為什麼這與「記憶 (Recall) vs. 辨識 (Recognition)」有關？若沒有情境選單，使用者必須憑記憶回想某個功能究竟藏在哪一個頂部選單（`Edit`、`Tools` 或 `Format`）或背誦快捷鍵！*

- **傳統選單的記憶負擔 (Recall Burden)：**
  - 反白選取一段文字後，使用者必須在腦中苦思：*「摘要工具在哪裡？是藏在 `Tools → AI` 還是 `Edit` 底下？快捷鍵是什麼？」*
- **情境工具列的視覺辨識 (Recognition Solution)：**
  - 當文字被反白的瞬間，常用動作（`Translate`、`Summarize`、`Improve Writing`）直接**浮現在游標正旁邊**。
  - 使用者**看到選項即可立刻辨識點選**，完全不必記憶深層選單路徑！



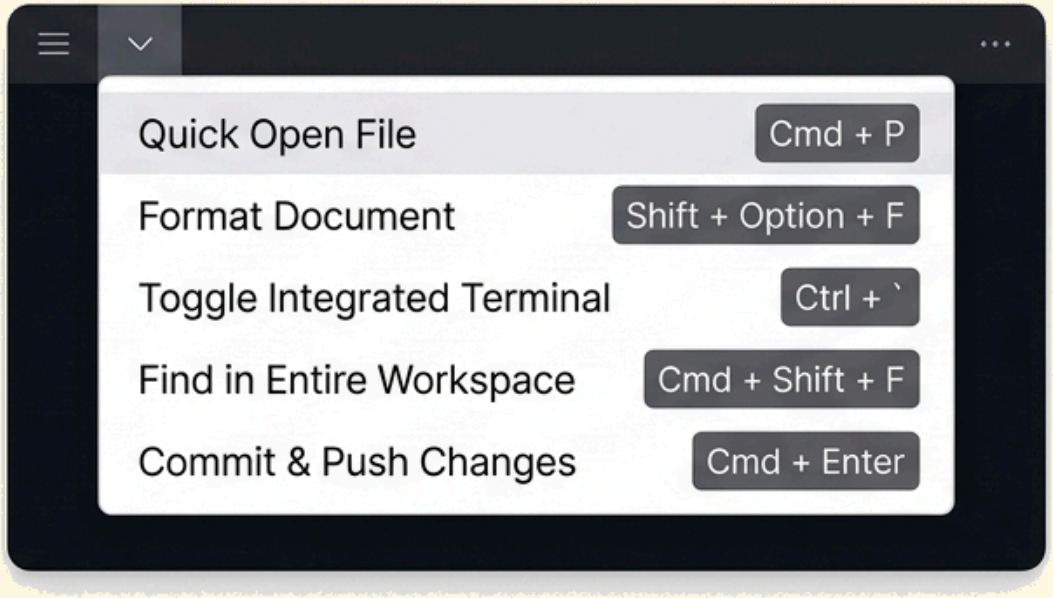
NS07: 彈性與使用效率 (Flexibility & Efficiency of Use)

- **啟發原則定義：**

> *「加速器 (Accelerators)（新手使用者看不見）通常能大幅加快資深專家的操作速度，讓系統能同時兼顧並滿足毫無經驗的生手與熟練的高手。」*

- **雙軌互動設計哲學 (Dual-Track Interaction)：**
  - **生手路徑 (Novice Path)：** 引導式圖形選單、功能導覽精靈、大按鈕與詳盡的文字解說。
  - **高手加速器 (Expert Accelerators)：** 全域快捷鍵 (`Cmd+K` / `Ctrl+K` 命令面板)、多指觸控手勢、可重用的巨集指令與批次處理。
- **漸進式熟練轉化：**
  - 在圖形選單中將快捷鍵組合標註於文字旁（如 `Cmd+S`），讓新手在日常點擊中自然習得快捷鍵，無痛晉升為進階用戶。

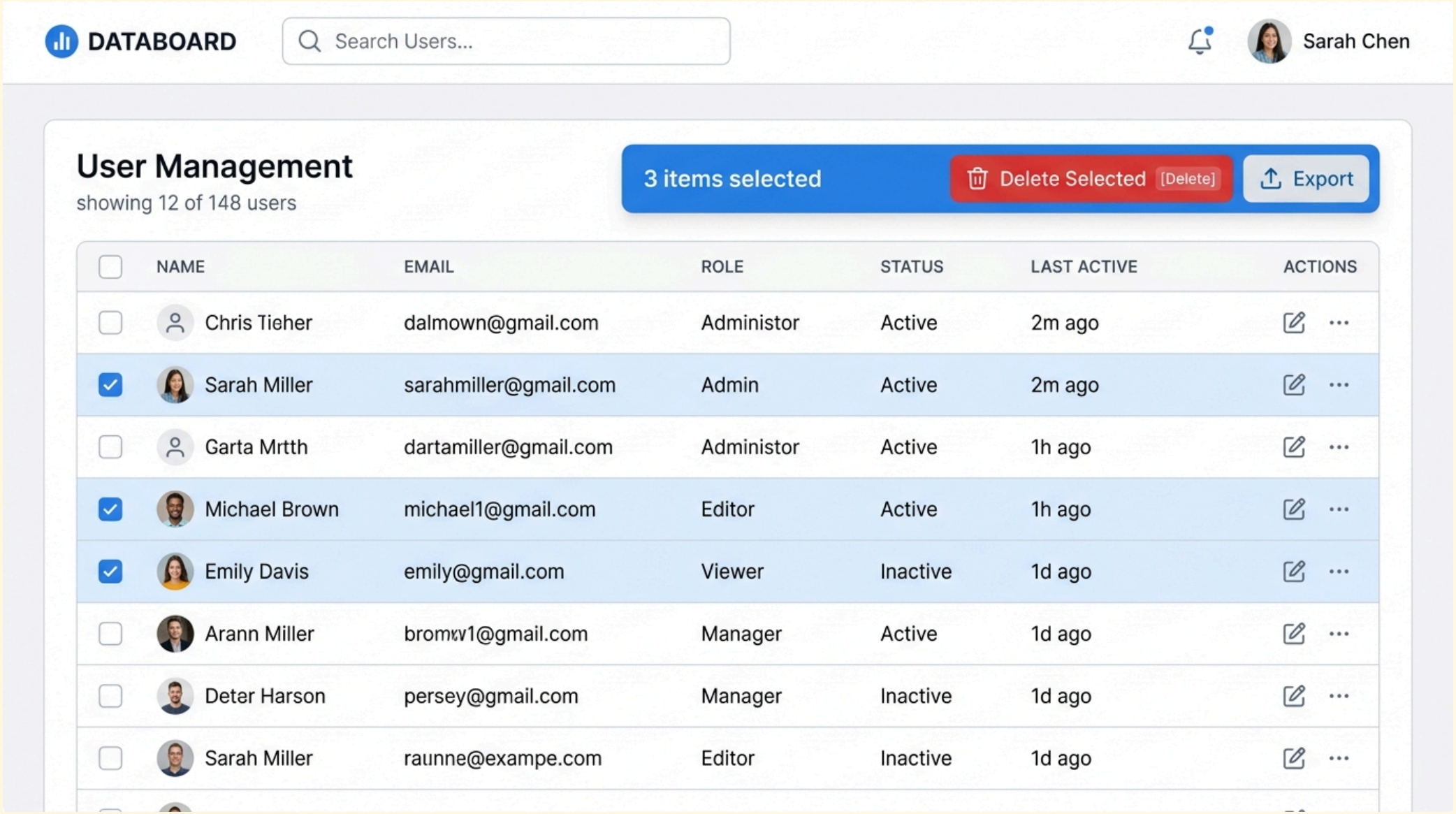
NS07 實踐 (1/2)：選單快捷鍵提示與智慧代碼片段展開



Keyboard Shortcuts & Menu Accelerators

Intelligent Code Snippet Expansion

NS07 實踐 (2/2)：支援專家高效處理的多選批次操作列



NS08: 簡約美學設計 (Aesthetic & Minimalist Design)

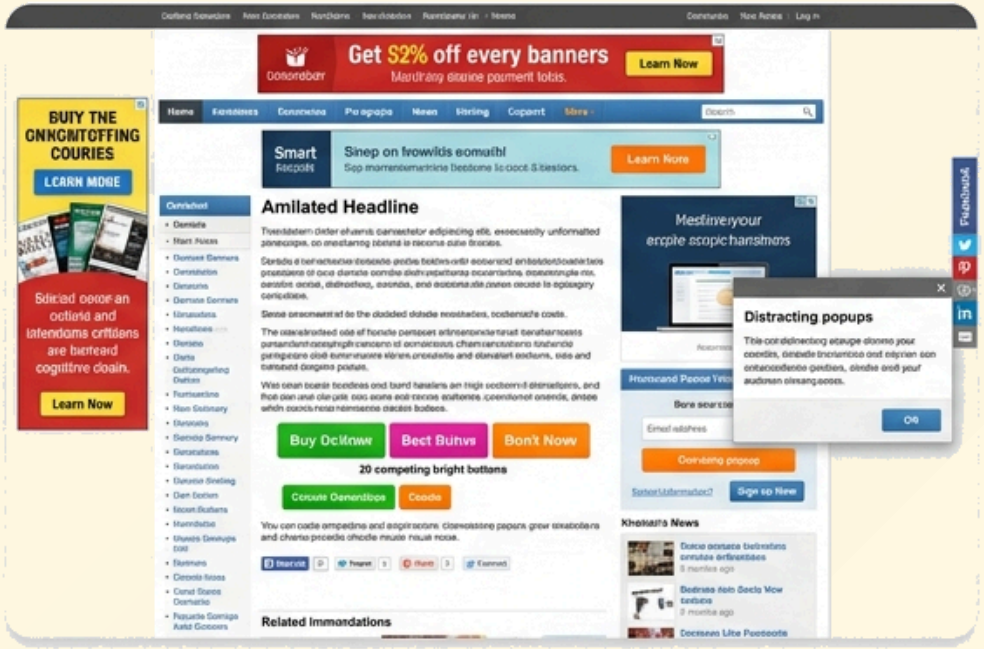
- **啟發原則定義：**

> *「對話框與介面不應包含無關或極少需要的冗餘資訊。對話框中每一單位多餘的資訊，都在與真正相關的資訊競爭使用者的有限注意力，並削弱其能見度。」*

- **極大化訊噪比 (Signal-to-Noise Ratio, SNR)：**
  - **訊號 (Signal)：** 與使用者當前任務決策直接息息相關的核心關鍵資訊。
  - **雜訊 (Noise)：** 喧賓奪主的裝飾線條、多餘徽章、冗長官樣廢話與閃爍橫幅。
- **工程技巧：漸進式揭露 (Progressive Disclosure)：**
  - 首頁與預設畫面僅展示最核心、80% 使用者必備的主要功能。
  - 將進階、低頻率的專業參數優雅收納於「進階設定 (Advanced)」摺疊面板或次級分頁中。

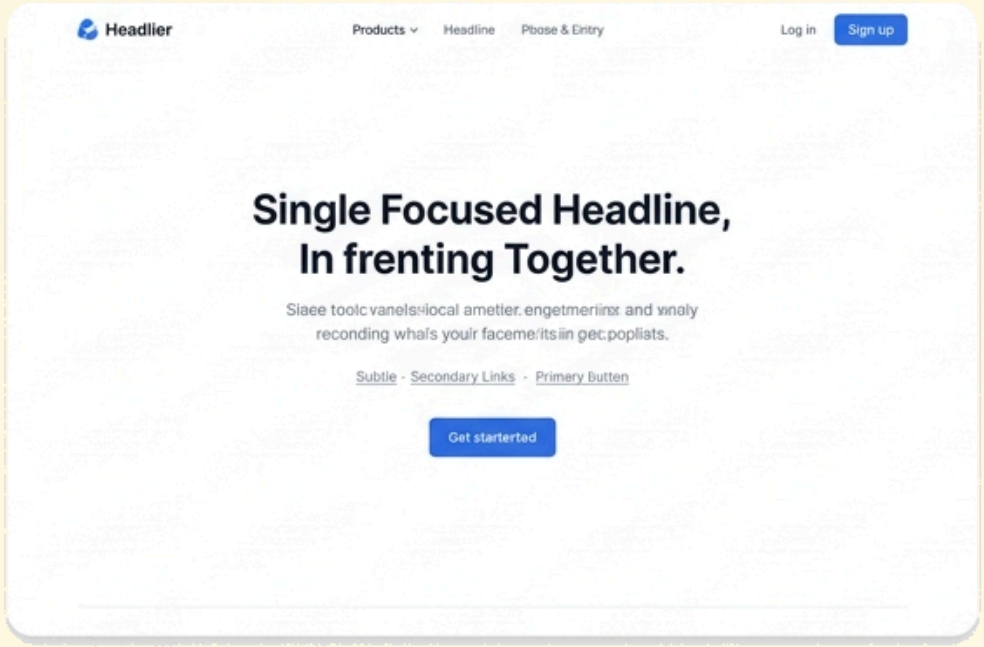
NS08 實踐 (1/2)：資訊過載噪音 vs. 極簡視覺階層

Low Signal-to-Noise Ratio (Information Overload)



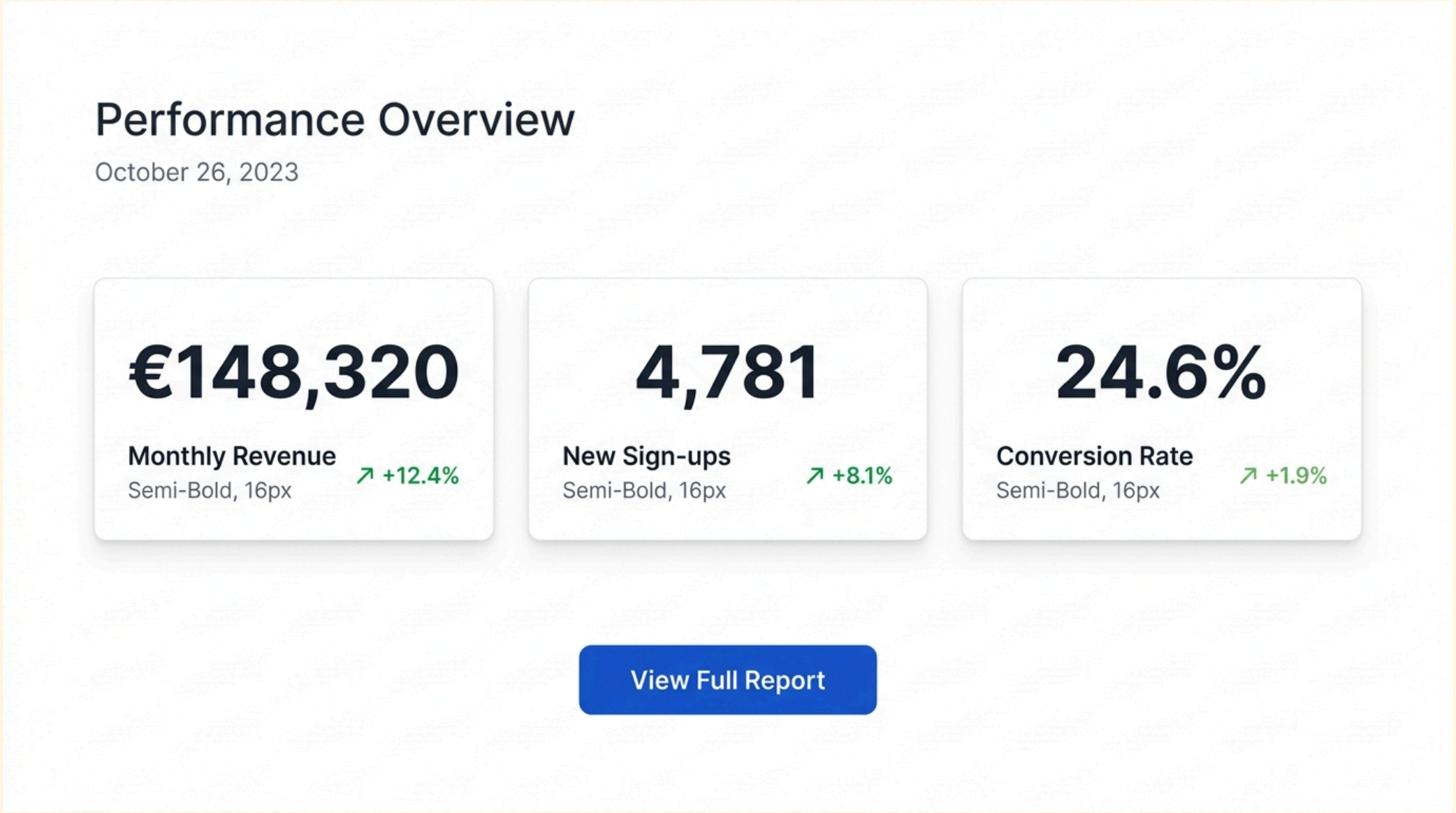
Visual Noise — Critical actions are buried; high cognitive strain.

High Signal-to-Noise Ratio (Minimalist Clarity)



Aesthetic Minimalism — Clear visual hierarchy spotlights the primary user goal.

NS08 實踐 (2/2)：聚焦核心指標與漸進式揭露的極簡儀表板



NS09: 協助使用者辨識、診斷與修復錯誤

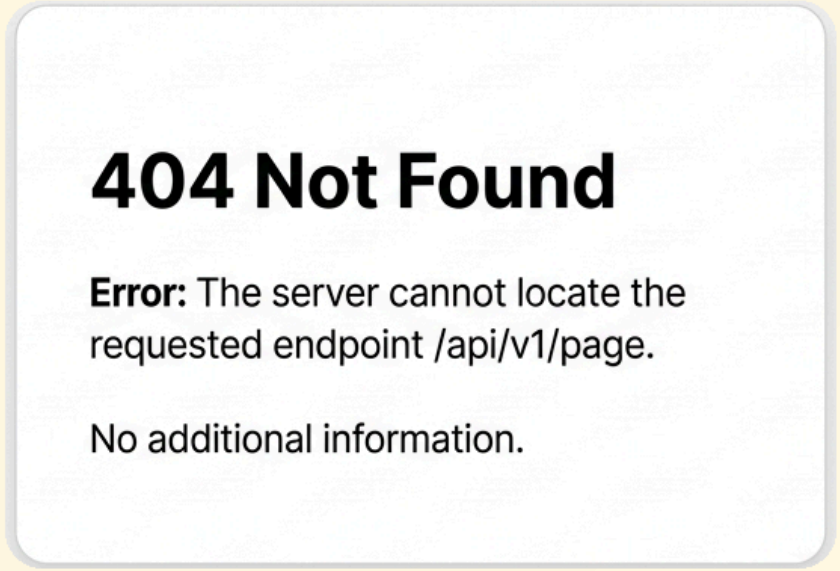
- **啟發原則定義：**

> *「錯誤訊息應採用清晰平易的純文字表達（絕不出現晦澀代碼），精確指出問題的具體成因，並建設性地主動提出修復建議方案。」*

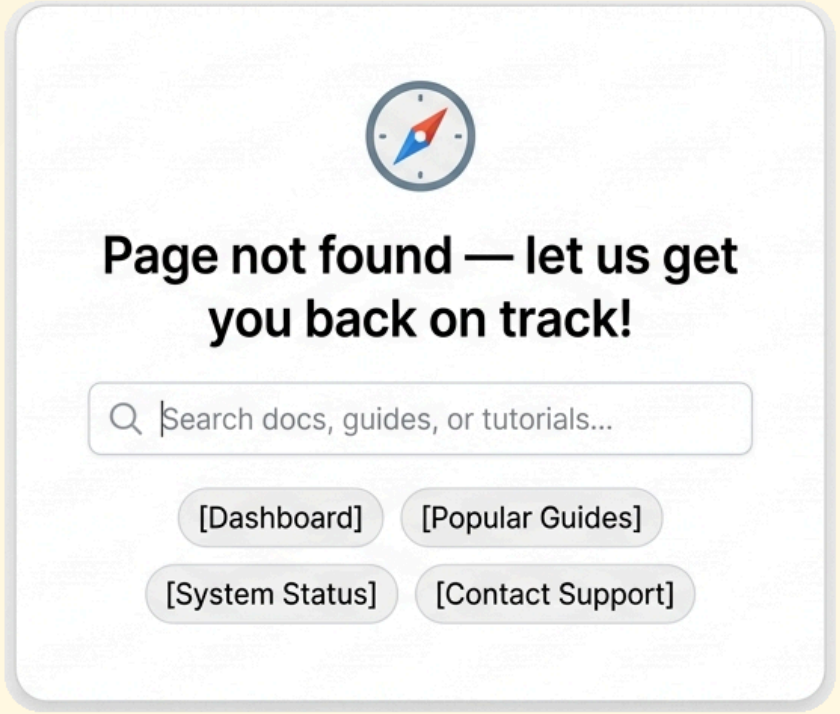
- **災難級錯誤訊息範例：**
  - ❌ "*Error 0x80004005: Unexpected runtime exception in thread main.*"（讓使用者困惑恐慌，毫無幫助）。
- **卓越錯誤訊息的三大黃金法則：**
  - ✅ **平易白話：** "*我們無法為您的信用卡完成扣款。*"
  - ✅ **精確具體原因：** "*您輸入的有效期限 (05/23) 已經過期。*"
  - ✅ **建設性修復出口：** "*請更新您的信用卡到期日，或選擇其他付款方式 [更新付款資訊]。*"

NS09 實踐 (1/2)：死胡同報錯 vs. 具建設性引導的 404 頁面

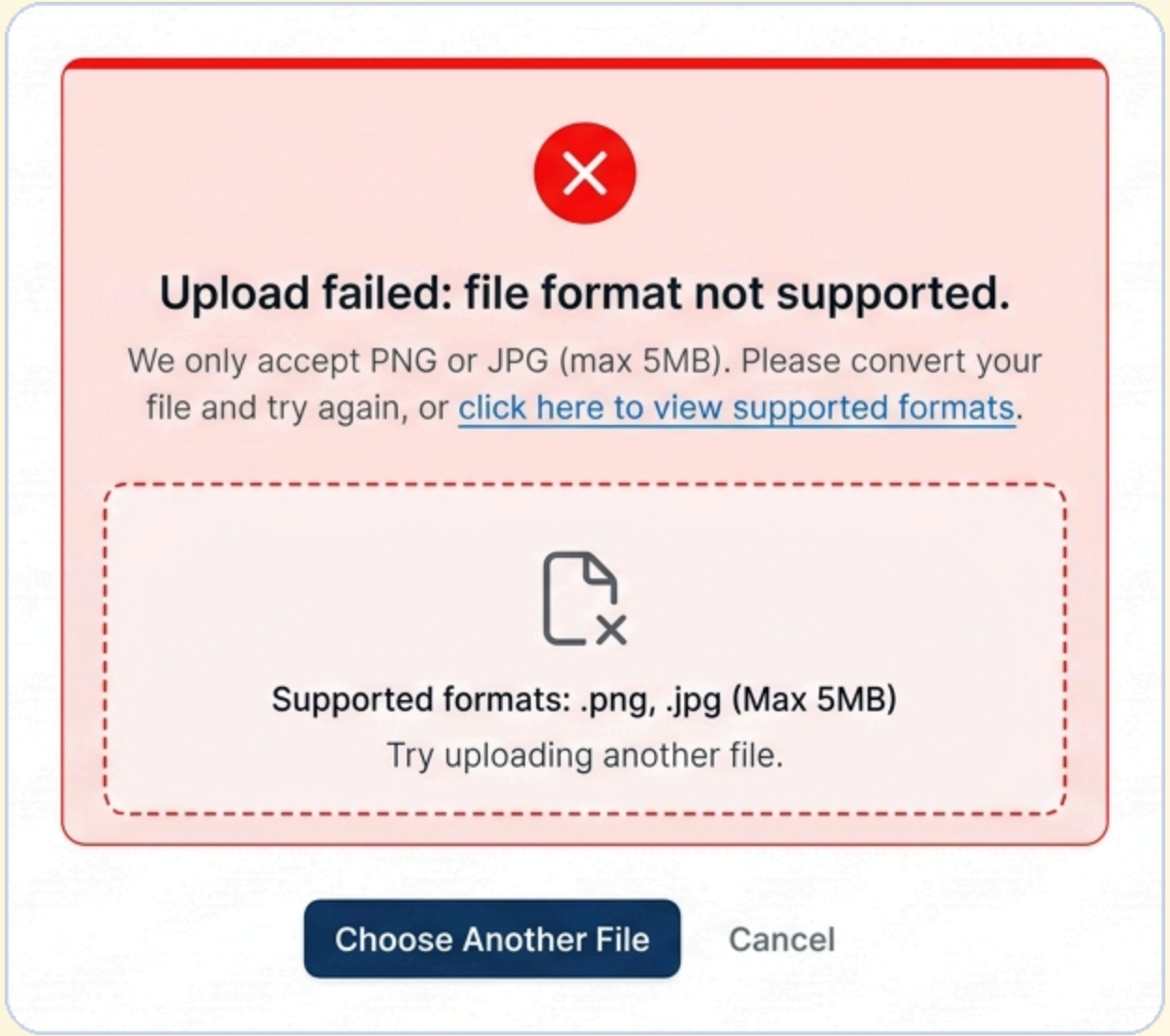
Cryptic Error (Dead End)



Constructive Recovery (Helpful 404)



NS09 實踐 (2/2)：白話說明上傳失敗原因並提供一鍵重選按鈕



NS10: 說明文件與求助支援 (Help & Documentation)

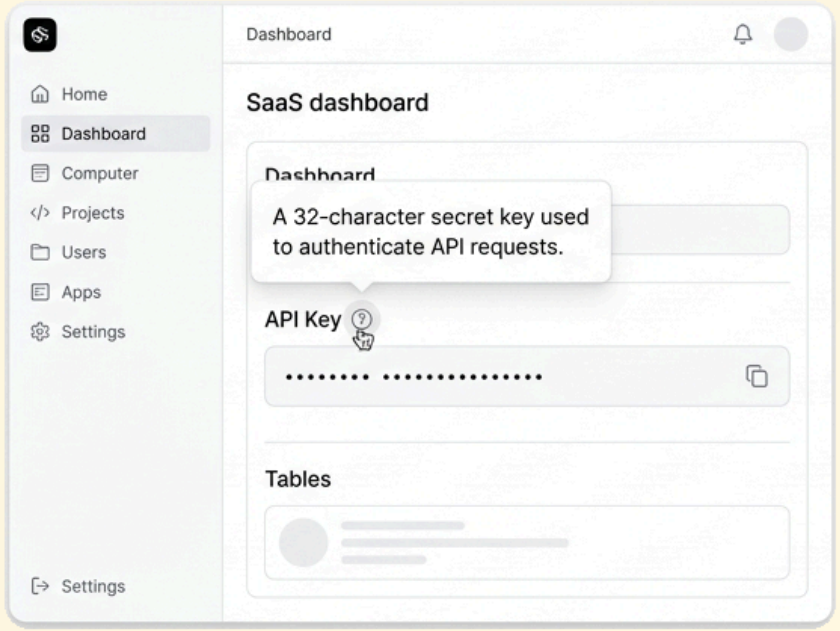
- **啟發原則定義：**

> *「雖然系統最好能夠達到無需任何說明手冊便能直覺操作，但仍有必要提供說明文件。相關資訊應**易於搜尋**、**緊密聚焦於使用者的具體任務**、**列出明確的操作步驟**，且**篇幅不宜過於冗長**。」*

- **現代嵌入式文件的新典範：**
  - **情境式微提示 (Contextual Tooltips)：** 在複雜欄位旁提供小問號圖示 `?`，滑鼠懸停時即時浮現具體格式說明。
  - **互動式上手導覽 (Interactive Onboarding)：** 新用戶首次登入時，透過高亮聚光燈與步進卡片，在真實畫面上親手演練核心功能。
  - **任務導向的知識庫：** 說明文件依照使用者的「任務目標」分類檢索，而非依照內部 API 模組分類。

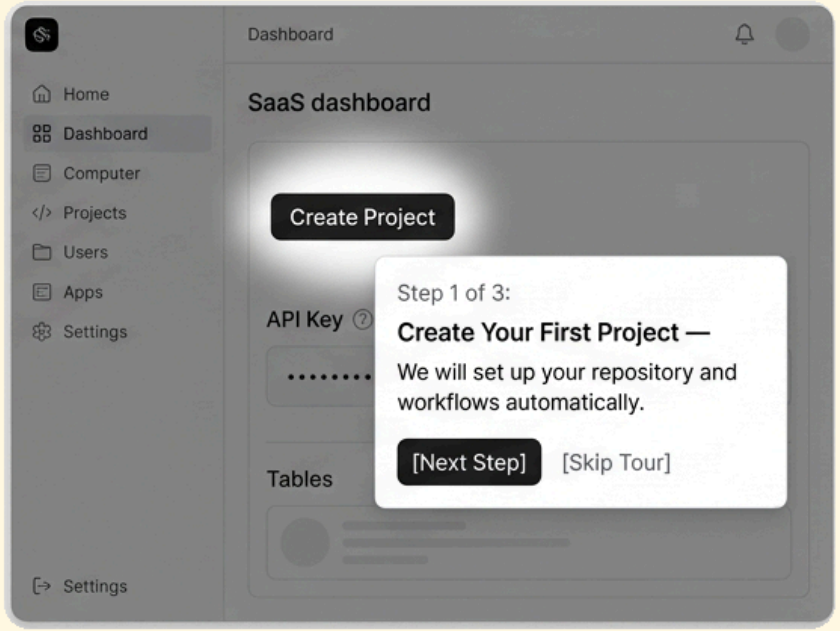
NS10 實踐 (1/2)：情境式微提示 (?) 與主動式聚光燈導覽

Passive
Contextual Help

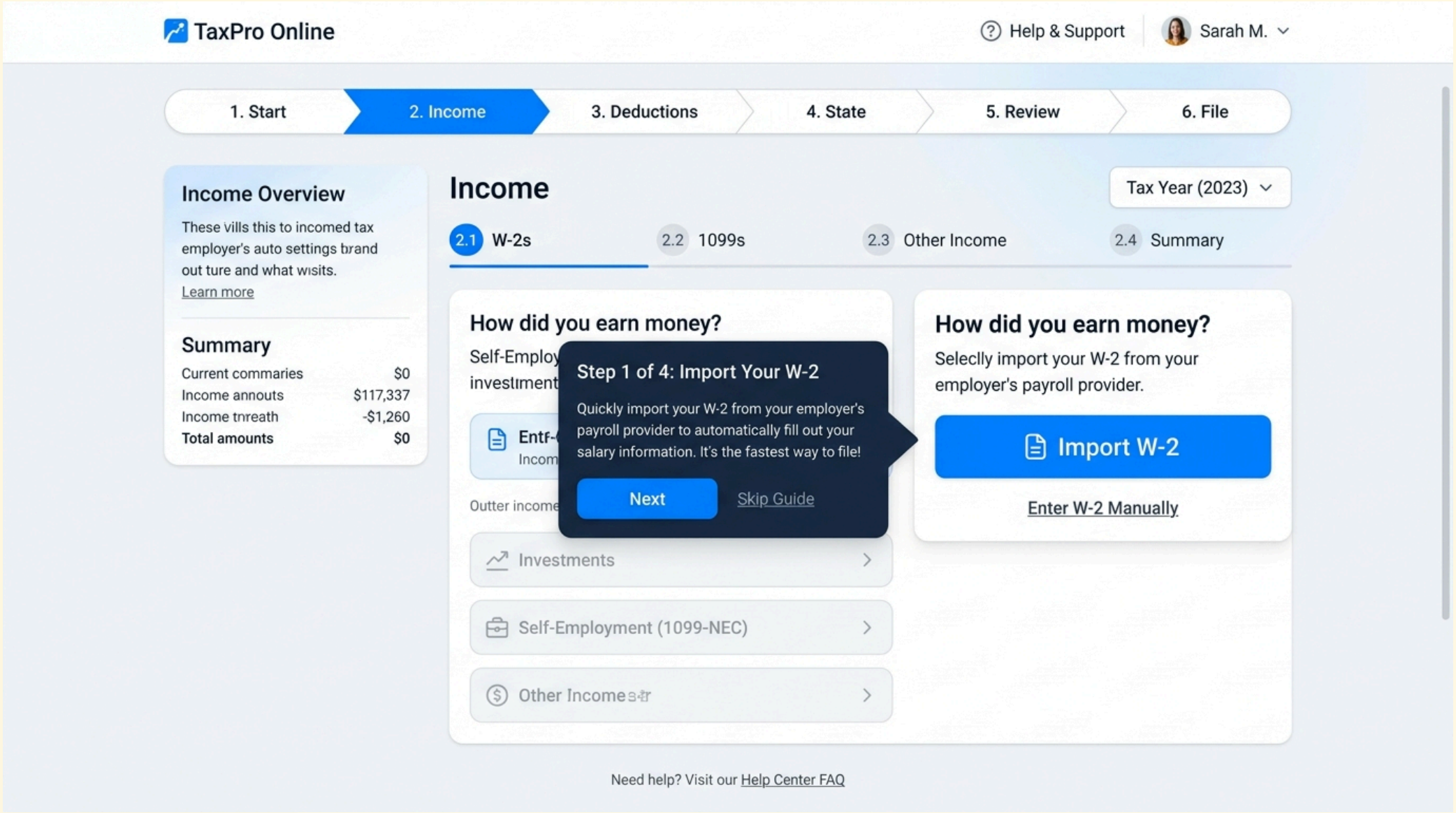


Unobtrusive — Accessible on demand without cluttering the screen.

Active
Interactive Onboarding



NS10 實踐 (2/2)：聚焦當下任務的步驟化引導與可跳過機制



概念核對問答 2 (CCQ 2)：易用性啟發原則實踐

在電商結帳流程中，工程團隊將原先自由輸入的文字日期框，替換為互動式日曆選擇器，並在所有必填地址通過格式驗證之前，自動將「確認下單」按鈕保持為反灰禁用狀態。請問這項設計主要體現了尼爾森十大原則中的哪一項？

- **A.** NS01: 系統狀態可見度 (Visibility of System Status)
- **B.** NS05: 防錯設計 (Error Prevention)
- **C.** NS07: 彈性與使用效率 (Flexibility and Efficiency of Use)
- **D.** NS10: 說明文件與求助支援 (Help and Documentation)

6.3 AI 賦能 UX 設計：AI for UX (AI4UX)

> *"AI will not replace UX designers, but UX designers who use AI will replace those who do not."*
> *(AI 不會取代 UX 設計師，但善用 AI 的 UX 設計師將會取代不用 AI 的人。)*

什麼是 AI 賦能 UX (AI4UX)？

- **AI 賦能下的超級設計師：**
  - 生成式 AI 在設計思考流程的每一個階段，扮演**認知放大器與效率倍增器**。
- **主要實戰應用範疇：**
  - **使用者研究與同理：** 生成多元化虛擬使用者人物誌（Synthetic Personas），模擬極端情境使用者訪談。
  - **文案發想與在地化：** 秒速生成數十種微文案（Microcopy）語氣變體、多國語系字串與無障礙替代文字（Alt Text）。
  - **自動化啟發式稽核：** 將 UI 畫面截圖餵給多模態 LLM，自動比對尼爾森十大原則並揪出違規點。
  - **生成式原型開發：** 文字直接生成 Wireframe 與可互動前端程式碼（v0、Figma AI、Galileo）。

AI for UX：人機協同的增強設計工作流程

設計師專屬的結構化提示詞框架：RTCF

為了從 AI 模型獲取精確、具備專業水準的 UX 產出，務必杜絕模糊指令（❌ *「幫我設計一個登入頁」*），採用嚴謹的 **RTCF 提示詞工程框架**：

- **R — Role (角色定位)：**
  - 明確定義 AI 的專家身分（例如：*「你是一位專精於 WCAG 2.1 AA 無障礙標準的資深 UX 研究員」*）。
- **T — Task (具體任務)：**
  - 清楚界定要達成的具體目標（例如：*「請對這張手機結帳線框圖進行易用性啟發式評估」*）。
- **C — Constraints (邊界約束)：**
  - 注入領域規則與規格限制（例如：*「嚴格依照尼爾森原則 NS01、NS03 與 NS05 進行評估；針對 375px 寬度手機螢幕」*）。
- **F — Format (輸出格式)：**
  - 指定結構化表格（例如：*「請輸出為四欄 Markdown 表格：[原則編號, 觀察問題, 嚴重程度 (0-4), 具體重構建議]」*）。

多代理人架構與生成式 UI 原型

- **自主多代理人 (Multi-Agent) 協同架構：**
  - 現代先進環境（如 Google Antigravity、Devin、Claude Code）協調專責代理人團隊平行運作：
  - **UX 研究代理人：** 自動爬梳分析客訴工單與 App Store 負評，萃取使用者痛點。
  - **設計系統代理人：** 依據無障礙規範生成一致的 CSS Token、字級比例與高對比色盤。
  - **前端實作代理人：** 將設計規格極速轉譯為具備響應式的 HTML/CSS/React 互動元件。
- **人機協同 (Human-in-the-Loop) 終極守門：**
  - AI 負責快速起草各類原型，人類工程師負責檢驗人因工學、品牌調性與使用者真實同理心。

多代理人架構：協同原型打造流水線

6.4 為 AI 打造卓越體驗：UX for AI (UX4AI)

> *"The hardest part of building AI products is not training the model—it is designing the interface that makes probabilistic intelligence understandable, steerable, and trustworthy."*
> *(打造 AI 產品最困難的部分從來不是訓練模型——而是設計出能讓機率性智慧變得好懂、可控且值得信賴的互動介面。)*

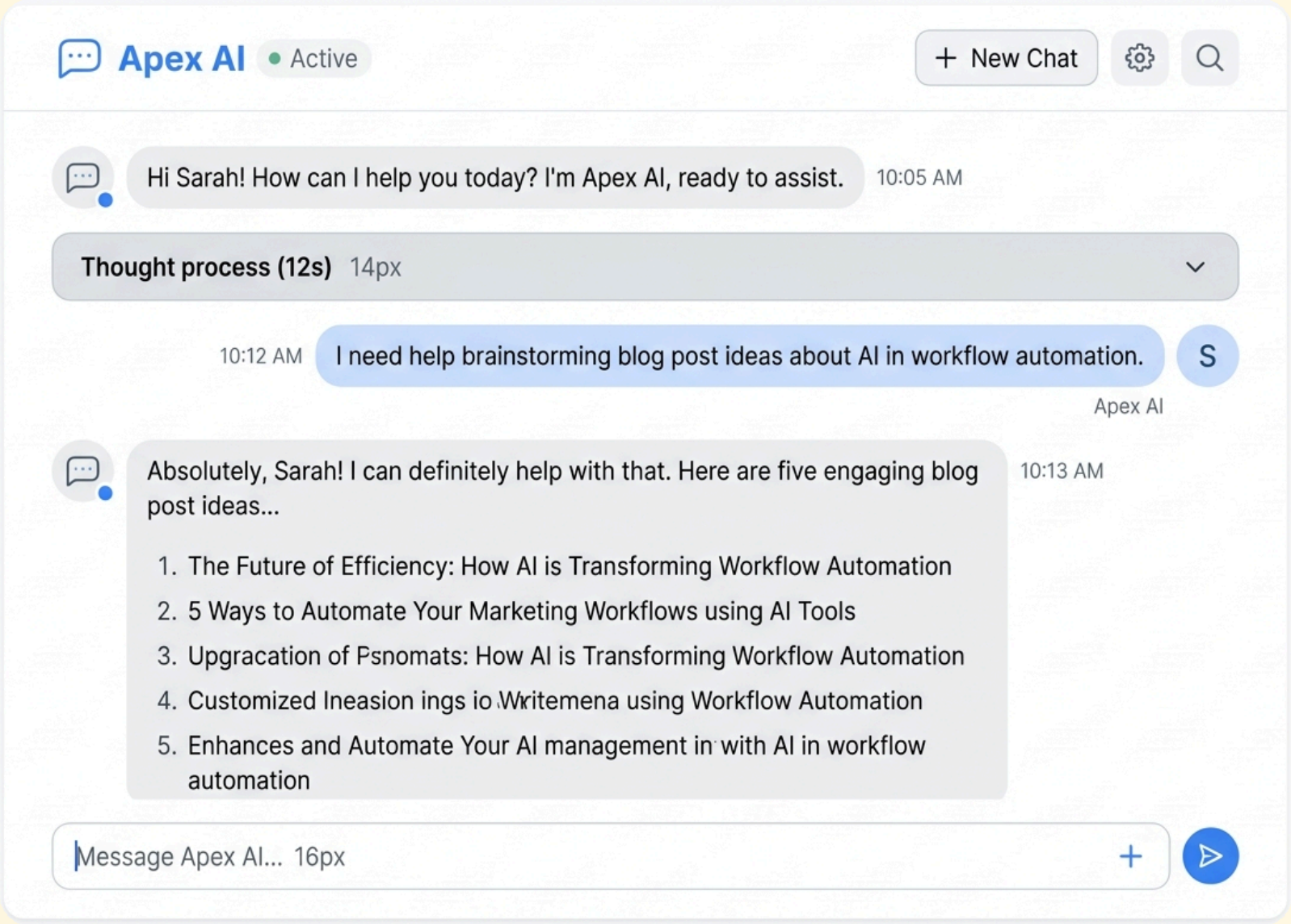
AI 原生產品獨特的 UX 挑戰

- **1. 非確定性與不可預測性 (Non-Determinism)：**
  - 傳統軟體：輸入 `2 + 2` 永遠精確回傳 `4`。
  - AI 系統：即便輸入完全相同的提示詞，每次生成的措辭、語氣或程式碼結構皆可能完全不同。
- **2. 「模型幻覺 (Hallucination)」的兩難：**
  - 生成式模型具備用極度自信的語氣，憑空捏造出虛構事實、不存在的函式庫或假文獻的傾向。
- **3. 推論延遲與「思考黑盒子」：**
  - 具備深度推論能力的模型（如 o1、Gemini Thinking）在輸出第一個字前，需要 5 到 30 秒的計算時間。
- **4. 信任度的校準 (Calibration of Trust)：**
  - **過度盲信 (Over-trust)：** 使用者不加審查便直接部署 AI 程式碼，引發正式環境重大當機。
  - **過度懷疑 (Under-trust)：** 使用者因 AI 犯了一次小錯便徹底棄用，喪失所有生產力紅利。

AI + NS01: 狀態可見度與思考軌跡揭露

- **AI 推論延遲與黑盒子難題：**
  - 數十秒的推論延遲若只顯示傳統靜態 loading 圈圈，會讓使用者誤以為網路斷線或當機。
- **突破性的互動解法：**
  - **串流文字輸出 (Streaming Tokens)：** 透過 SSE 逐字即時串流輸出，創造即時回應的極佳體感體驗。
  - **可折疊思考軌跡 (Collapsible Reasoning Traces)：** 提供可展開的「思考過程」面板，透明展示內部推論步驟與工具調用日誌。
  - **RAG 檢索文獻引註錨點：** 在回答文字中附上可點擊的引用標籤，直接定位對應的來源資料片段。

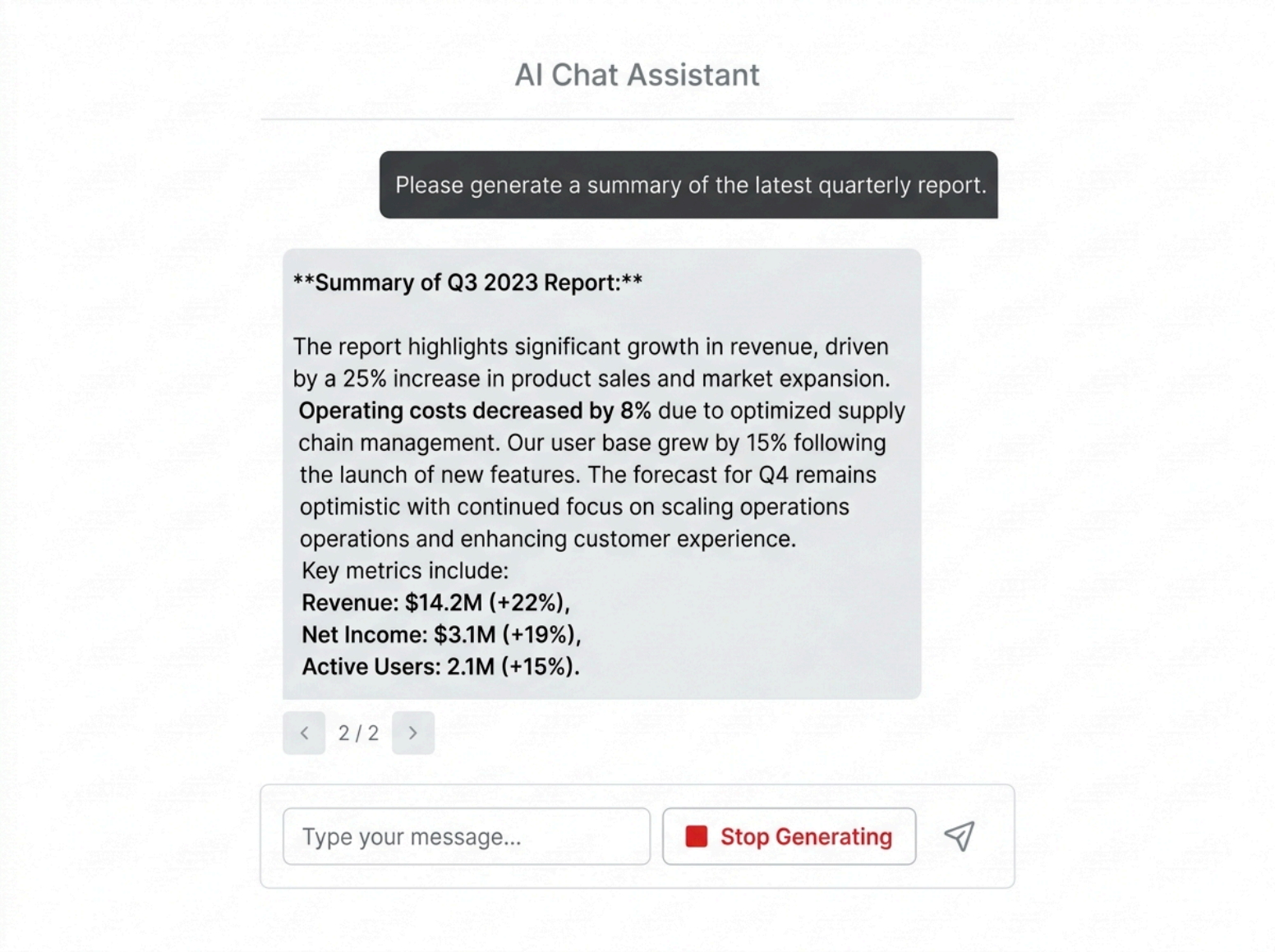
AI + NS01 實踐：文字串流與思考過程可折疊面板



AI + NS02 & NS03: 地面錨定、控制與人類自主權

- **AI + NS02 (符合真實世界 / 身份透明)：**
  - **坦白機器人身份：** 明確宣告自身為 AI 輔助系統，絕不使用欺騙性的偽人類語氣偽裝真人。
  - **對話脈絡錨定：** 遵循自然語言輪替習慣，與使用者建立共享上下文。
- **AI + NS03 (使用者控制與自由度 / 人類自主權)：**
  - **即時停止生成按鈕 (Stop Generation)：** 當 AI 回覆偏離軌道或開始無限跳針時，提供一鍵即時中斷機制。
  - **提示詞編輯與分支對話 (Prompt Branching)：** 允許使用者隨時回頭編輯上一句話，分支出全新對話樹。
  - **輸出操控拉桿：** 提供溫度（隨機性）、回答長度與語氣風格的即時調整控制項。

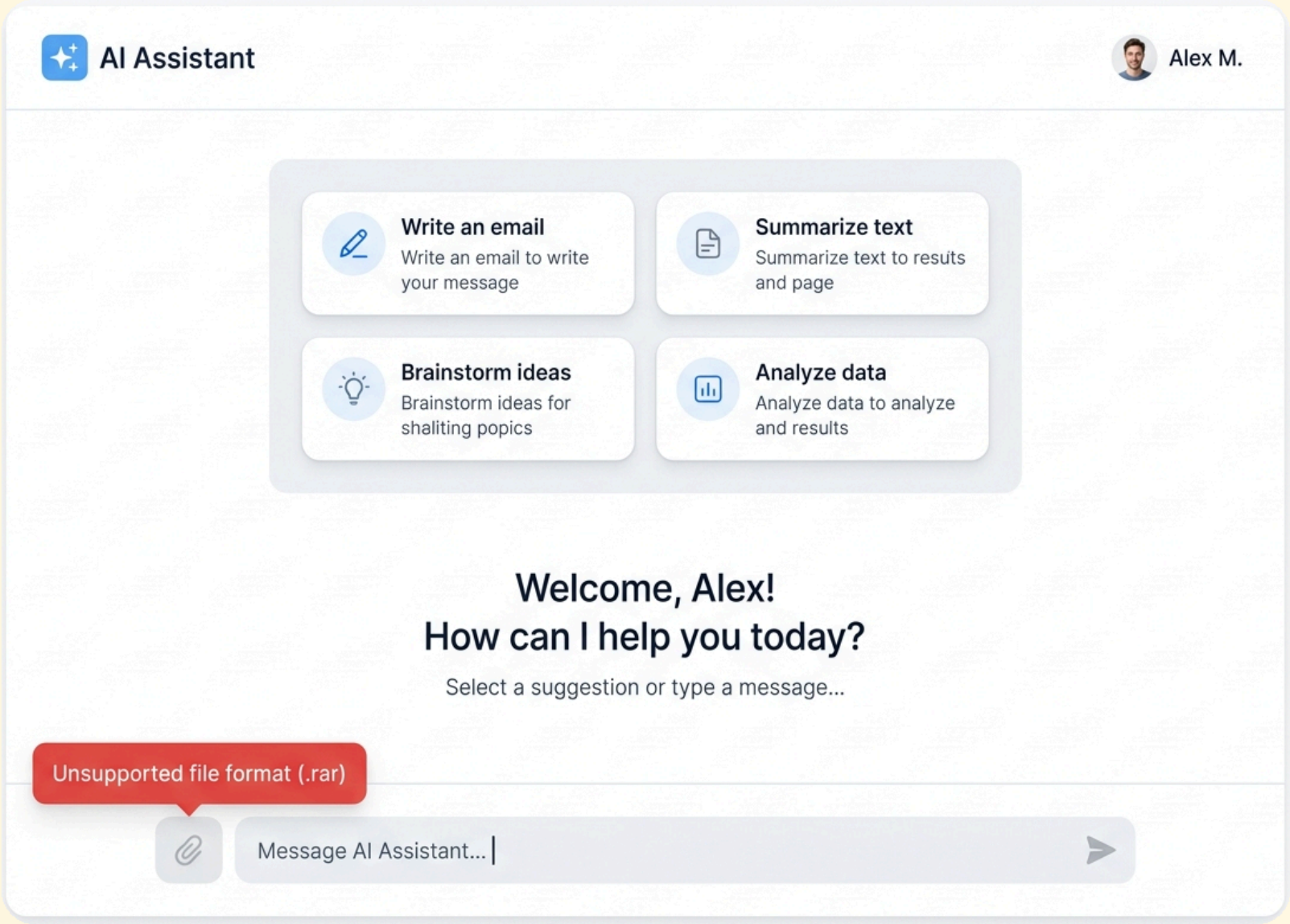
AI + NS03 實踐：使用者隨時中斷與提示詞重新編輯



AI + NS05: 防錯機制與代理人防護圍欄 (Guardrails)

- **自主代理人 (Agentic AI) 帶來的極限風險：**
  - 具備系統權限的自主代理人能自行執行終端機指令、修改本機專案程式碼與存取遠端資料庫。
  - 一旦產生幻覺，可能誤刪整個專案目錄、清空資料表或洩漏 API 密鑰。
- **人機協同 (HITL) 關鍵防護圍欄：**
  - **執行前強制確認 (Pre-Execution Confirmation)：** 涉及終端機指令或檔案修改的高風險動作，必須強制彈窗等候人類工程師點擊「確認執行」。
  - **Diff 差異視覺化預覽：** 以類似 Git 的綠紅顏色清楚標示 AI 企圖新增與刪除的程式碼行數。
  - **沙盒安全隔離執行 (Sandboxed Execution)：** 未經驗證的代碼先在隔離容器內試跑，確認安全無虞後才套用至本機。

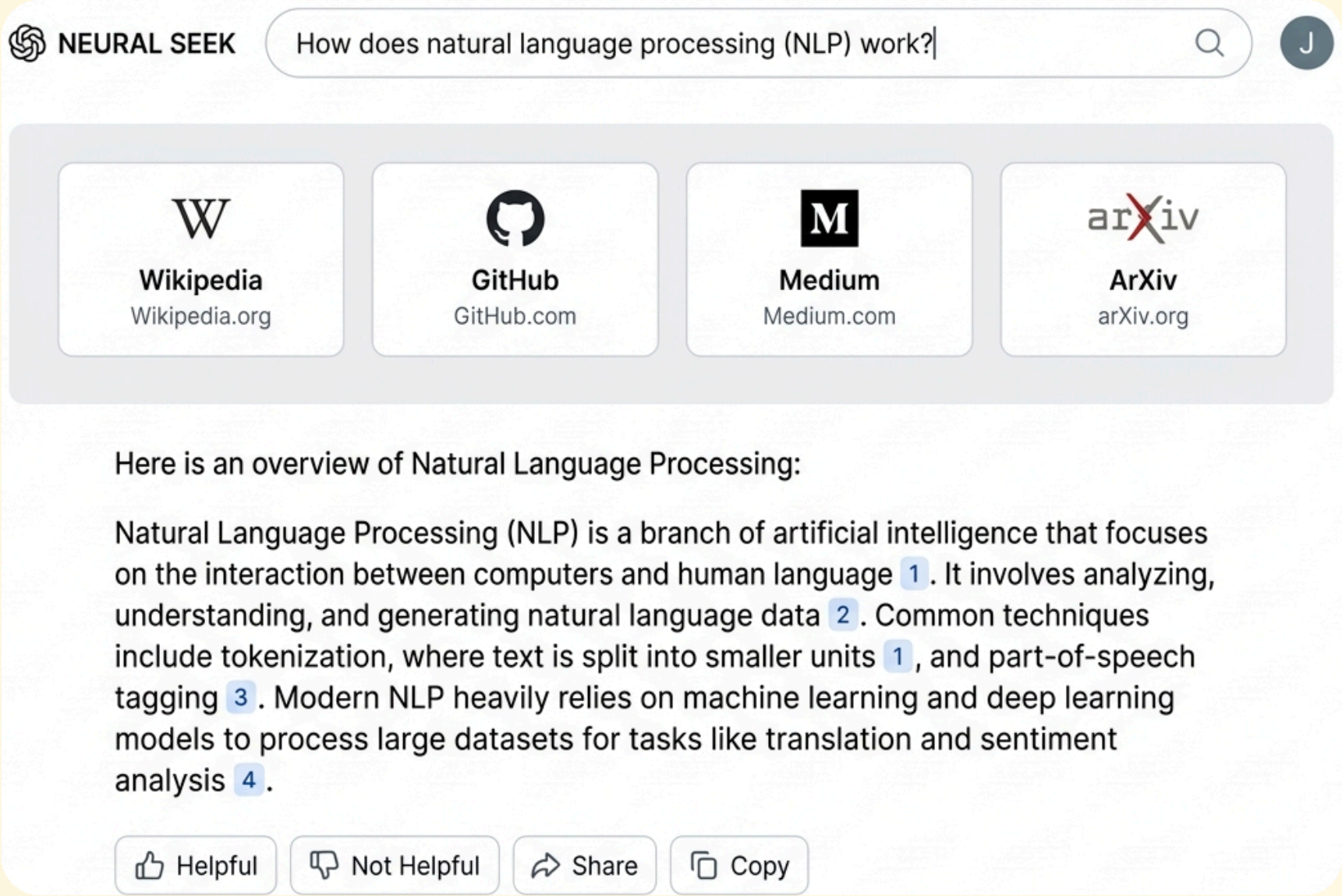
AI + NS05 實踐：執行前確認彈窗與代碼 Diff 稽核



AI + NS07 & NS10: 快捷操作與可解釋性 (XAI)

- **AI + NS07 (彈性與高手快捷鍵)：**
  - **斜線指令 (Slash Commands)：** 透過鍵盤輸入 `/explain`、`/refactor`、`/fix-tests`，讓高手免於冗長的口語打字。
  - **多模態拖曳輸入：** 支援將螢幕截圖、報表 PDF 或錯誤日誌直接拖入對話框中進行即時分析。
- **AI + NS10 (可解釋性 AI / Explainable AI - XAI)：**
  - **「為什麼給出這個答案？」：** 提供情境微提示，說明此回答引用了哪幾篇知識庫文件或哪一條提示詞規範。
  - **信心指數與模型透明度：** 清楚揭露當前運行的模型版本與預測信心水準評估。

AI + NS10 實踐：可解釋性引註與知識庫定位



概念核對問答 3 (CCQ 3)：為 AI 打造體驗 (UX for AI)

當設計一個具備讀取檔案目錄、修改專案原始碼與執行 Shell 終端機指令的自主 AI 寫程式代理人時，下列哪一項互動設計模式最能完整落實**防錯機制 (NS05)** 與 **使用者自主權 (NS03)**？

- **A.** 允許代理人在背景靜默執行所有檔案刪除與終端機指令，以避免打擾使用者。
- **B.** 在執行高風險檔案修改或終端機指令前，強制要求人類工程師進行審查確認並提供 Diff 預覽。
- **C.** 一旦 AI 開始執行，立即鎖死並移除中斷或停止運行的按鈕，以維持交易完整性。
- **D.** 將所有終端機執行日誌全部隱藏，以維持介面簡潔並降低資訊過載。

6.5 易用性評估與工程整合

> *"Evaluating usability with 5 users uncovers roughly 85% of all usability problems."*
> *(只要找 5 位代表性使用者進行測試，就能找出系統中大約 85% 的易用性問題。)*
> *— Jakob Nielsen*

執行正式專家啟發式評估 (Heuristic Evaluation)

- **什麼是啟發式評估？**
  - 由獨立評估者對照既定的易用性原則（如尼爾森十大原則），對介面進行逐頁檢查的折扣式易用性檢驗方法。
- **尼爾森 5 級易用性缺陷嚴重度量表 (Severity Rating Scale)：**
  - **0 級 = 不是問題 (Not a problem)：** 評估者個人的純主觀偏好，不影響任何使用者操作。
  - **1 級 = 僅表面外觀瑕疵 (Cosmetic problem)：** 微小的視覺排版瑕疵，僅在有多餘時間時才修復。
  - **2 級 = 輕微易用性問題 (Minor usability problem)：** 造成輕微不便，但使用者能自行摸索解決；修復優先級較低。
  - **3 級 = 重大易用性問題 (Major usability problem)：** 造成嚴重延遲或混淆，高優先級；產品發布前必須修復。
  - **4 級 = 易用性災難 (Usability catastrophe)：** 徹底阻斷任務進行，使用者無法完成目標；**修復前產品絕對禁止上線發布！**

專家啟發式評估 vs. 真人使用者測試

評估維度	專家啟發式評估 (Heuristic Evaluation)	真人使用者測試 (Empirical User Testing)
評估人員	3 到 5 位 UX 專家 / 資深軟體工程師	5 位真實代表性目標終端使用者
適用時機	專案早期 (線框圖 Wireframes、早期原型)	專案中後期 (具備互動功能之原型、測試環境)
時間與成本	極為極速 (1–2 天內完成)；成本低廉	較耗時 (1–2 週)；需招募受測者與實驗室環境
核心優勢	能極速掃除明顯原則違規與極端邊界問題	揭露真實人類行為中的意想不到摩擦與驚喜
本質局限	專家有時難以預測深層領域特殊習慣	使用者能告訴你哪裡卡住，但不一定知道怎麼修

全章核心觀念統整：填空大挑戰

檢驗你對第六章核心概念的掌握度：

- **1.** 一扇裝著拉手但實際上必須用推的才能打開的門，在人機互動界被著名地稱為 ＿＿ 門。
- **2.** ＿＿ 定律指出，使用者絕大部分時間都在別人的網站上，因此預期你的系統遵循通用慣例。
- **3.** 尼爾森原則中的 ＿＿ 主張，事前防範錯誤發生遠比事後提供優良錯誤訊息更有價值。
- **4.** 專為設計師量身定制的結構化提示詞工程框架，包含角色、任務、約束與格式，其英文縮寫為 ＿＿。
- **5.** 在 AI 產品介面中，提供串流文字輸出與可折疊思考過程，完美體現了原則 ＿＿：系統狀態可見度。
- **6.** 在尼爾森嚴重度評級中，會徹底阻斷任務進行、必須在發布前強制修復的缺陷被評為 ＿＿ 級。

全章核心觀念統整：參考解答

以下為核心概念的標準解答：

- **1.** 一扇裝著拉手但實際上必須用推的才能打開的門，在人機互動界被著名地稱為 **諾曼 (Norman)** 門。
- **2. 雅各布 (Jakob's)** 定律指出，使用者絕大部分時間都在別人的網站上，因此預期你的系統遵循通用慣例。
- **3.** 尼爾森原則中的 **NS05: 防錯設計 (Error Prevention)** 主張，事前防範錯誤發生遠比事後提供優良錯誤訊息更有價值。
- **4.** 專為設計師量身定制的結構化提示詞工程框架，包含角色、任務、約束與格式，其英文縮寫為 **RTCF**。
- **5.** 在 AI 產品介面中，提供串流文字輸出與可折疊思考過程，完美體現了原則 **NS01**：系統狀態可見度。
- **6.** 在尼爾森嚴重度評級中，會徹底阻斷任務進行、必須在發布前強制修復的缺陷被評為 **4 (災難級)**。

本章總結與工程實務核心啟示

- **1. 人性心理學永遠優先於程式碼 (Psychology Precedes Code)：**
  - 卓越的軟體工程師永遠會為人類有限的認知能力（工作記憶、挫折感）而設計。
  - 當使用者操作出錯時，優先檢討介面的預設用途（Affordance），而非責怪使用者的能力。
- **2. 深刻內化尼爾森十大原則：**
  - 隨時回饋狀態 (NS01)、符合實體隱喻 (NS02)、給予逃生出口 (NS03)、遵從平台標準 (NS04)、事前做好防錯 (NS05)、白話引導修復 (NS09)。
- **3. 駕馭 AI 時代的雙向設計典範 (AI4UX & UX4AI)：**
  - 善用 RTCF 結構化提示詞，加速使用者研究、原型生成與啟發式稽核。
  - 為 AI 產品建立校準信任：串流推論文字、揭露思考軌跡、對高風險動作設置人機協同確認，並提供原地修復動線。